

第3章 PCI总线的数据交换

PCI Agent 设备之间以及 HOST 处理器和 PCI Agent 设备之间可以使用存储器读写和 I/O 读写等总线事务进行数据传送。在大多数情况下，PCI 桥不直接与 PCI 设备或者 HOST 主桥进行数据交换，而仅转发来自 PCI Agent 设备或者 HOST 主桥的数据。

PCI Agent 设备间的数据交换并不是本章讨论的重点。本章更侧重讲述 PCI Agent 设备使用 DMA 机制读写主存储器的数据，以及 HOST 处理器如何访问 PCI 设备的 BAR 空间。本章还将使用一定的篇幅讨论在 PCI 总线中与 Cache 相关的总线事务，并在最后介绍预读机制。

3.1 PCI 设备 BAR 空间的初始化

在 PCI Agent 设备进行数据传送之前，系统软件需要初始化 PCI Agent 设备的 BAR0~5 寄存器和 PCI 桥的 Base、Limit 寄存器。系统软件使用 DFS 算法对 PCI 总线进行遍历时，完成这些寄存器的初始化，即分配这些设备在 PCI 总线域的地址空间。当这些寄存器初始化完毕后，PCI 设备可以使用 PCI 总线地址进行数据传递。

值得注意的是，PCI Agent 设备的 BAR0~5 寄存器和 PCI 桥的 Base 寄存器保存的地址都是 PCI 总线地址。而这些地址在处理器系统的存储器域中具有映像，如果一个 PCI 设备的 BAR 空间在存储器域中没有映像，处理器将不能访问该 PCI 设备的 BAR 空间。

如上文所述，处理器通过 HOST 主桥将 PCI 总线域与存储器域隔离。当处理器访问 PCI 设备的地址空间时，需要首先访问该设备在存储器域中的地址空间，并通过 HOST 主桥将这个存储器域的地址空间转换为 PCI 总线域的地址空间之后，再使用 PCI 总线事务将数据发送到指定的 PCI 设备中。

PCI 设备访问存储器域的地址空间，即进行 DMA 操作时，也是首先访问该存储器地址空间所对应的 PCI 总线地址空间，之后通过 HOST 主桥将这个 PCI 总线地址空间转换为存储器地址空间，再由 DDR 控制器对存储器进行读写访问。

不同的处理器系统采用不同的机制实现存储器域和 PCI 总线域的转换。如 PowerPC 处理器使用 Outbound 寄存器组实现存储器域到 PCI 总线域间的转换，并使用 Inbound 寄存器组实现 PCI 总线域到存储器域间的转换。

而 x86 处理器没有这种地址空间域的转换机制，因此从 PCI 设备的角度来看，PCI 设备可以直接访问存储器地址；从处理器的角度来看，处理器可以直接访问 PCI 总线地址空间。但是读者需要注意，在 x86 处理器的 HOST 主桥中仍然有存储器域与 PCI 总线域这个概念。只是在 x86 处理器的 HOST 主桥中，存储器域的存储器地址与 PCI 总线地址相等，这种“简单相等”也是一种映射关系。

3.1.1 存储器地址与 PCI 总线地址的转换

下面根据 PowerPC 和 x86 处理器的主桥，抽象出一个虚拟的 HOST 主桥，并以此为例讲述 PCI Agent 设备之间以及 PCI Agent 设备与主存储器间的数据传送过程。

假设在一个 32 位处理器中，存储器域的 0xF000-0000~0xF7FF-FFFF（共 128MB）这段物理地址空间与 PCI 总线的地址空间存在映射关系。

当处理器访问这段存储器地址空间时，HOST 主桥会认领这个存储器访问，并将该存储器访问使用的物理地址空间转换为 PCI 总线地址空间，并与 0x7000-0000~0x77FF-FFFF 这段 PCI 总线地址空间对应。

为简化起见，假定在存储器域中只映射了 PCI 设备的存储器地址空间，而不映射 PCI 设备的 I/O 地址空间。而 PCI 设备的 BAR 空间使用 0x7000-0000~0x77FF-FFFF 这段 PCI 总线域的存储器地址空间。

在这个 HOST 主桥中，存储器域与 PCI 总线域的映射关系如图 3-1 所示。

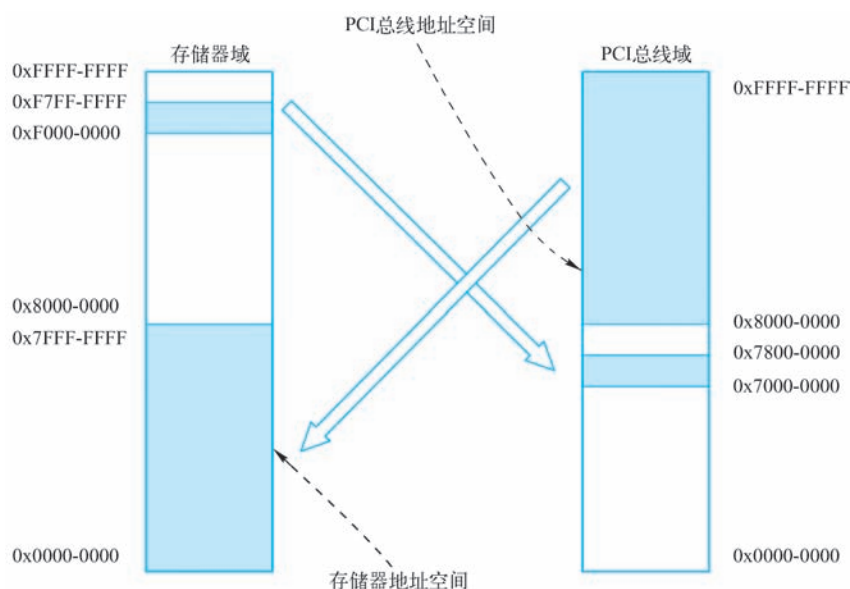


图 3-1 存储器域与 PCI 总线域的映射关系

当 PCI 设备使用 DMA 机制访问存储器域地址空间时，处理器系统同样需要将存储器域的地址空间反向映射到 PCI 总线地址空间。假设在一个处理器系统中，主存储器大小为 2 GB，其在存储器域的地址范围为 0x0000-0000~0x7FFF-FFFF，而这段地址在 PCI 总线域中对应的“PCI 总线地址空间”为 0x8000-0000~0xFFFF-FFFF。

PCI 设备进行 DMA 操作时，必须使用 0x8000-0000~0xFFFF-FFFF 这段 PCI 总线域的地址，HOST 主桥才能认领这个 PCI 总线事务，并将这个总线事务使用的 PCI 总线地址转换为存储器地址，并与 0x0000-0000~0x7FFF-FFFF 这段存储器区域进行数据传递。

在一个实际的处理器系统中，很少有系统软件采用这样的方法，实现存储器域与 PCI 总线域之间的映射，“简单相等”还是最常用的映射方法。本章采用图 3-1 的映射关系，虽然增加了映射复杂度，却便于读者深入理解存储器域到 PCI 总线域之间的映射关系。下面将以这种映射关系为例，详细讲述 PCI 设备 BAR0~5 寄存器的初始化。

3.1.2 PCI 设备 BAR 寄存器和 PCI 桥 Base、Limit 寄存器的初始化

PCI 桥的 Base、Limit 寄存器保存“该桥所管理的 PCI 子树”的存储器或者 I/O 空间的

基地址和长度。值得注意的是，PCI 桥也是 PCI 总线上的一个设备，在其配置空间中也有 BAR 寄存器，本节不对 PCI 桥 BAR 寄存器进行说明，因为在多数情况下透明桥并不使用其内部的 BAR 寄存器。下面以图 3-2 所示的处理器系统为例说明上述寄存器的初始化过程，该处理器系统使用的存储器域与 PCI 总线域的映射关系如图 3-1 所示。

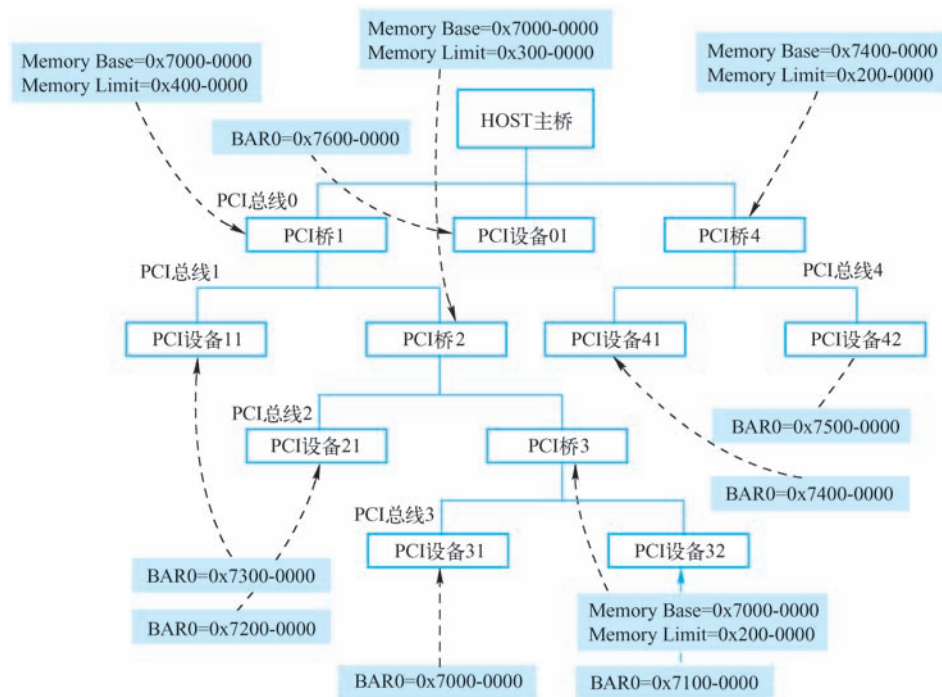


图 3-2 BAR 寄存器的初始化

在 PCI 设备的 BAR 寄存器中，包含该设备使用的 PCI 总线域的地址范围。在 PCI 设备的配置空间中，共有 6 个 BAR 寄存器，因此一个 PCI 设备最多可以使用 6 组 32 位的 PCI 总线地址空间，或者 3 组 64 位的 PCI 总线地址空间。这些 BAR 空间可以保存 PCI 总线域的存储器地址空间或者 I/O 地址空间，目前多数 PCI 设备仅使用存储器地址空间。而在通常情况下，一个 PCI 设备使用 2~3 个 BAR 寄存器就足够了。

为简化起见，首先假定在图 3-2 中所示的 PCI 总线树中，所有 PCI Agent 设备只使用了 BAR0 寄存器，其申请的数据空间大小为 16MB（即 0x1000000B）而且不可预读，而且 PCI 桥不占用 PCI 总线地址空间，即 PCI 桥不含有 BAR 空间。并且假定当前 HOST 主桥已经完成了对 PCI 总线树的编号。

根据以上假设，该 PCI 总线树的遍历过程如下所示。

1) 系统软件根据 DFS 算法，首先寻找到第一组 PCI 设备，分别为 PCI 设备 31 和 PCI 设备 32^①，并根据这两个 PCI 设备需要的 PCI 空间大小，从 PCI 总线地址空间中（0x7000-0000~

① HOST 主桥下的第一个桥片是 PCI 桥 1，PCI 桥 1 下的第一个桥片是 PCI 桥 2，而 PCI 桥 2 下的第一个桥片是 PCI 桥 3，因而第一组 PCI 设备为 PCI 总线 3 下的 PCI 设备。不同的系统软件查找第一组 PCI 设备的方法不同，Linux 认为第一组 PCI 设备为 PCI 总线 0 下的 PCI 设备。

0x77FF-FFFF) 为这两个 PCI 设备的 BAR0 寄存器分配基地址, 分别为 0x7000- 0000 和 0x7100- 0000。

2) 当系统软件完成 PCI 总线 3 下所有设备的 BAR 空间的分配后, 将初始化 PCI 桥 3 的配置空间。这个桥片的 Memory Base 寄存器保存其下所有 PCI 设备使用的“PCI 总线域地址空间的基地址”, 而 Memory Limit 寄存器保存其下 PCI 设备使用的“PCI 总线域地址空间的大小”。系统软件将 Memory Base 寄存器赋值为 0x7000- 0000, 而将 Memory Limit 寄存器赋值为 0x200- 0000。

3) 系统软件回溯到 PCI 总线 2, 并找到 PCI 总线 2 上的 PCI 设备 21, 并将 PCI 设备 21 的 BAR0 寄存器赋值为 0x7200- 0000。

4) 完成 PCI 总线 2 的遍历后, 系统软件初始化 PCI 桥 2 的配置寄存器, 将 Memory Base 寄存器赋值为 0x7000- 0000, Memory Limit 寄存器赋值为 0x300- 0000。

5) 系统软件回溯到 PCI 总线 1, 并找到 PCI 设备 11, 并将这个设备的 BAR0 寄存器赋值为 0x7300- 0000。并将 PCI 桥 1 的 Memory Base 寄存器赋值为 0x7000- 0000, Memory Limit 寄存器赋值为 0x400- 0000。

6) 系统软件回溯到 PCI 总线 0, 并在这条总线上发现另外一个 PCI 桥, 即 PCI 桥 4。并使用 DFS 算法继续遍历 PCI 桥 4。首先系统软件将遍历 PCI 总线 4, 并发现 PCI 设备 41 和 PCI 设备 42, 并将这两个 PCI 设备的 BAR0 寄存器分别赋值为 0x7400- 0000 和 0x7500- 0000。

7) 系统软件初始化 PCI 桥 4 的配置寄存器, 将 Memory Base 寄存器赋值为 0x7400- 0000, Memory Limit 寄存器赋值为 0x200- 0000。系统软件再次回到 PCI 总线 0, 这一次系统软件没有发现新的 PCI 桥, 于是将初始化这条总线上的所有 PCI 设备。

8) PCI 总线 0 上只有一个 PCI 设备, 即 PCI 设备 01。系统软件将这个设备的 BAR0 寄存器赋值为 0x7600- 0000, 并结束整个 DFS 遍历过程。

3.2 PCI 设备的数据传递

PCI 设备的数据传递使用地址译码方式, 当一个存储器读写总线事务到达 PCI 总线时, 在这条总线上的所有 PCI 设备将进行地址译码, 如果当前总线事务使用的地址在某个 PCI 设备的 BAR 空间中, 该 PCI 设备将使能 DEVSEL#信号, 认领这个总线事务。

如果 PCI 总线上的所有设备都不能通过地址译码, 认领这个总线事务时, 这条总线的“负向译码”设备将认领这个总线事务, 如果在这条 PCI 总线上没有“负向译码”设备, 该总线事务的发起者将使用 Master Abort 总线周期结束当前 PCI 总线事务。

3.2.1 PCI 设备的正向译码与负向译码

如上文所述, PCI 设备使用“地址译码”方式接收存储器读写总线请求。在 PCI 总线中定义了两种“地址译码”方式, 一种是正向译码, 另一种是负向译码。

下面仍以图 3-2 所示的处理器系统为例, 说明数据传送使用的寻址方法。当 HOST 主桥通过存储器或者 I/O 读写总线事务访问其下所有 PCI 设备时, PCI 总线 0 下的所有 PCI 设备都将对出现在地址周期中的 PCI 总线地址进行译码。如果这个地址在某个 PCI 设备的 BAR 空间中命中时, 这个 PCI 设备将接收这个 PCI 总线请求。这个过程也被称为 PCI 总线的正向

译码，这种方式也是大多数 PCI 设备所采用的译码方式。

但是在 PCI 总线上的某些设备，如 PCI-to-(E)ISA 桥并不使用正向译码接收来自 PCI 总线的请求，PCI-to-ISA 桥在处理器系统中的位置如图 1-1 所示。PCI 总线 0 上的总线事务在三个时钟周期后，没有得到任何 PCI 设备响应时（即总线请求的 PCI 总线地址不在这些设备的 BAR 空间中），PCI-to-ISA 桥将被动地接收这个数据请求。这个过程被称为 PCI 总线的负向译码。可以进行负向译码的设备也被称为负向译码设备。

在 PCI 总线中，除了 PCI-to-(E)ISA 桥可以作为负向译码设备，PCI 桥也可以作为负向译码设备，但是 PCI 桥并不是在任何时候都可以作为负向译码设备。在绝大多数情况下，PCI 桥无论是处理“来自上游总线”，还是处理“来自下游总线”的总线事务时，都使用正向译码方式，但是在某些特殊应用中，PCI 桥也可以作为负向译码设备。

笔记本在连接 Dock 插座时，也使用了 PCI 桥。因为在大多数情况下，笔记本与 Dock 插座是分离使用的，而且 Dock 插座上连接的设备多为慢速设备，此时用于连接 Dock 插座的 PCI 桥使用负向译码。Dock 插座在笔记本系统中的位置如图 3-3 所示。

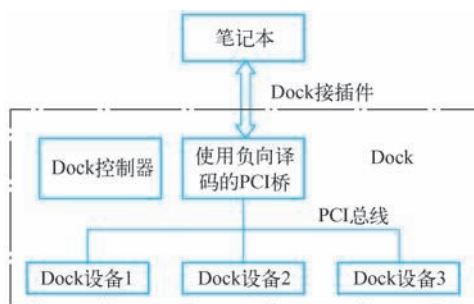


图 3-3 Dock 插座与笔记本的连接关系

当笔记本与 Dock 建立连接之后，如果处理器需要访问 Dock 中的外部设备时，Dock 中的 PCI 桥将首先使用负向译码方式接收 PCI 总线事务，之后将这个 PCI 总线事务转发到 Dock 的 PCI 总线中，再访问相应的 PCI 设备。

在 Dock 中使用负向译码 PCI 桥的优点是，该桥管理的设备并不参与处理器系统对 PCI 总线的枚举过程。当笔记本插入到 Dock 之后，系统软件并不需要重新枚举 Dock 中的设备并为这些设备分配系统资源，而仅需要使用负向译码 PCI 桥管理好其下的设备即可，从而极大降低了 Dock 对系统软件的影响。

当 HOST 处理器访问 Dock 中的设备时，负向译码 PCI 桥将首先接管这些存储器读写总线事务，然后发送到 Dock 设备中。值得注意的是，在许多笔记本的 Dock 实现中，并没有使用负向译码 PCI 桥，而使用 PCI-to-ISA 桥。

PCI 总线规定使用负向译码的 PCI 桥，其 Base Class Code 寄存器为 0x06，Sub Class Code 寄存器为 0x04，而 Interface 寄存器为 0x01；使用正向译码方式的 PCI 桥的 Interface 寄存器为 0x00。系统软件（E²PROM）在初始化 Interface 寄存器时务必注意这个细节。

综上所述，在 PCI 总线中有两种负向译码设备，PCI-to-E(ISA)桥和 PCI 桥。但 PCI 桥并非在任何时候都是负向译码设备，只有 PCI 桥连接 Dock 插座时，PCI 桥的 Primary Bus 才使用负向译码方式。而这个 PCI 桥的 Secondary Bus 在接收 Dock 设备的请求时仍然使用正向译码方式。

PCI 桥使用的正向译码方式与 PCI 设备使用的正向译码方式有所不同。如图 3-4 所示, 当一个总线事务是从 PCI 桥的 Primary Bus 到 Secondary Bus 时, PCI 桥使用的正向译码方式与 PCI 设备使用的方式类似。如果该总线事务使用的地址在 PCI 桥任意一个 Memory Base 窗口^①命中时, 该 PCI 桥将使用正向译码方式接收该总线事务, 并根据实际情况决定是否将这个总线事务转发到 Secondary Bus。

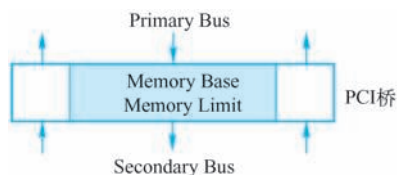


图 3-4 PCI 桥使用的正向译码方式

当一个总线事务是从 PCI 桥的 Secondary Bus 到 Primary Bus 时, 如果该总线事务使用的地址没有在 PCI 桥所有的 Memory Base 窗口命中, 表明当前总线事务不是访问该 PCI 桥管理的 PCI 子树中的设备, 因此 PCI 桥将接收当前总线事务, 并根据实际情况决定是否将这个总线事务转发到 Primary Bus。

以图 3-2 为例, 当 PCI 设备 11 访问主存储器空间时, 首先将存储器读写总线事务发送到 PCI 总线 1 上, 而这个存储器地址显然不会在 PCI 总线 1 的任何 PCI 设备的 BAR 空间中, 此时 PCI 桥 1 将认领这个 PCI 总线的数据请求, 并将这个总线事务转发到 PCI 总线 0 上。最后 HOST 主桥将接收这个总线事务, 并将 PCI 总线地址转换为存储器域的地址, 与主存储器进行读写操作。

值得注意的是, PCI 总线并没有规定 HOST 主桥使用正向还是负向译码方式接收这个存储器读写总线事务, 但是绝大多数 HOST 主桥使用正向译码方式接收来自下游的存储器读写总线事务。在 PowerPC 处理器中, 如果当前存储器读写总线事务使用的地址在 Inbound 窗口内, HOST 主桥将接收这个总线事务, 并将其转换为存储器域的读写总线事务, 与主存储器进行数据交换。

3.2.2 处理器到 PCI 设备的数据传送

下面以图 3-2 所示的处理器系统为例, 说明处理器向 PCI 设备 11 进行存储器写的数据传送过程。处理器向 PCI 设备进行读过程与写过程略有区别, 因为存储器写使用 Posted 方式, 而存储器读使用 Non-Posted 方式, 但是存储器读使用的地址译码方式与存储器写类似, 因此本节对处理器向 PCI 设备进行存储器读的过程不做进一步介绍。

PCI 设备 11 在 PCI 总线域的地址范围是 0x7300-0000~0x73FF-FFFF。这段空间在存储器域中对应的地址范围是 0xF300-0000~0xF3FF-FFFF。下面假设处理器使用存储器写指令, 访问 0xF300-0008 这个存储器地址, 其步骤如下。

- 1) 存储器域将 0xF300-0008 这个地址发向 HOST 主桥, 0xF000-0000~0xF7FF-FFFF 这

^① PCI 桥除了具有 Memory Base 窗口外, 还有 I/O Base 窗口和 Prefetchable Memory Base 窗口。

段地址已经由 HOST 主桥映射到 PCI 总线域地址空间，所以 HOST 主桥认为这是一个对 PCI 设备的访问。因此 HOST 主桥将首先接管这个存储器写请求。

2) HOST 主桥将存储器域的地址 0xF300- 0008 转换为 PCI 总线域的地址 0x7300- 0008，并通过总线仲裁获得 PCI 总线 0 的使用权，启动 PCI 存储器写周期，并将这个存储器写总线事务发送到 PCI 总线 0 上。值得注意的是，这个存储器读写总线事务使用的地址为 0x7300- 0008，而不是 0xF300- 0008。

3) PCI 总线 0 的 PCI 桥 1 发现 0x7300- 0008 在自己管理的地址范围内，于是接管这个存储器写请求，并通过总线仲裁逻辑获得 PCI 总线 1 的使用权，并将这个请求转发到 PCI 总线 1 上。

4) PCI 总线 1 的 PCI 设备 11 发现 0x7300- 0008 在自己的 BAR0 寄存器中命中，于是接收这个 PCI 写请求，并完成存储器写总线事务。

3.2.3 PCI 设备的 DMA 操作

下面以图 3-2 所示的处理器系统为例，说明 PCI 设备 11 向存储器进行 DMA 写的数据传送过程。PCI 设备的 DMA 写使用 Posted 方式而 DMA 读使用 Non-Posted 方式。本节不介绍 PCI 设备进行 DMA 读的过程，而将这部分内容留给读者分析。

假定 PCI 设备 11 需要将一组数据发送到 0x1000- 0000~0x1000-FFFF 这段存储器域的地址空间中。由上文所述，存储器域的 0x0000- 0000~0x7FFF-FFFF 这段存储器空间与 PCI 总线域的 0x8000- 0000~0xFFFF-FFFF 这段 PCI 总线地址空间对应。

PCI 设备 11 并不能直接操作 0x1000- 0000~0x1000-FFFF 这段存储器域的地址空间，PCI 设备 11 需要对 PCI 总线域的地址空间 0x9000- 0000~0x9000-FFFF 进行写操作，因为 PCI 总线地址空间 0x9000- 0000~0x9000-FFFF 已经被 HOST 主桥映射到 0x1000- 0000~0x1000-FFFF 这段存储器域。这个 DMA 写具体的操作流程如下。

1) 首先 PCI 设备 11 通过总线仲裁逻辑获得 PCI 总线 1 的使用权，之后将存储器写总线事务发送到 PCI 总线 1 上。值得注意的是，这个存储器写总线事务的目的地址是 PCI 总线域的地址空间 0x9000- 0000~0x9000-FFFF，这个地址是主存储器在 PCI 总线域的地址映像。

2) PCI 总线 1 上的设备将进行地址译码，确定这个写请求是不是发送到自己的 BAR 空间，在 PCI 总线 1 上的设备除了 PCI 设备 11 之外，还有 PCI 桥 2 和 PCI 桥 1。

3) 首先 PCI 桥 1、2 和 PCI 设备 11 对这个地址同时进行正向译码。PCI 桥 1 发现这个 PCI 地址并不在自己管理的 PCI 总线地址范围之内，因为 PCI 桥 1 所管理的 PCI 总线地址空间为 0x7000- 0000~0x73FF-FFFF。此时 PCI 桥 1 将接收这个存储器写总线事务，因为 PCI 桥 1 所管理的 PCI 总线地址范围并不包含当前存储器写总线事务的地址，所以其下所有 PCI 设备都不可能接收这个存储器写总线事务。

4) PCI 桥 1 发现自己并不能处理当前这个存储器写总线事务，则将这个存储器写总线事务转发到上游总线。PCI 桥 1 首先通过总线仲裁逻辑获得 PCI 总线 0 的使用权后，然后将这个总线事务转发到 PCI 总线 0。

5) HOST 主桥发现 0x9000- 0000~0x9000-FFFF 这段 PCI 总线地址空间与存储器域的存储器地址空间 0x1000- 0000~0x1000-FFFF 对应，于是将这段 PCI 总线地址空间转换成为存储器域的存储器地址空间，并完成对这段存储器的写操作。

6) 存储器控制器将从 HOST 主桥接收数据, 并将其写入主存储器。PCI 设备间的数据传递与 PCI 设备到存储器的数据传送大体类似。我们以 PCI 设备 11 将数据传递到 PCI 设备 42 为例说明这个转递过程。我们假定 PCI 设备 11 将一组数据发送到 PCI 设备 42 的 PCI 总线地址 0x7500-0000~0x7500-FFFF 这段地址空间中。这个过程与 PCI 设备 11 将数据发送到存储器的第 1~5 步基本类似, 只是第 5、6 步不同。PCI 设备 11 将数据发送到 PCI 设备 42 的第 5、6 步如下所示。

7) PCI 总线 0 发现其下的设备 PCI 桥 4 能够处理来自 PCI 总线 0 的数据请求, 则 PCI 桥 4 将接管这个 PCI 写请求, 并通过总线仲裁逻辑获得 PCI 总线 4 的使用权, 之后将这个存储器写请求发向 PCI 总线 4。此时 HOST 主桥不会接收当前存储器写总线事务, 因为 0x7500-0000~0x7500-FFFF 这段地址空间并不是 HOST 主桥管理的地址范围。

8) PCI 总线 4 的 PCI 设备 42 将接收这个存储器写请求, 并完成这个 PCI 存储器写请求总线事务。

PCI 总线树内的数据传送始终都在 PCI 总线域中进行, 不存在不同域之间的地址转换, 因此 PCI 设备 11 向 PCI 设备 42 进行数据传递时, 并不会进行 PCI 总线地址空间到存储器地址空间的转换。

3.2.4 PCI 桥的 Combining、Merging 和 Collapsing

由上所述, PCI 设备间的数据传递有时将通过 PCI 桥。在某些情况下, PCI 桥可以合并一些数据传递, 以提高数据传递的效率。PCI 桥可以采用 Combining、Merging 和 Collapsing 三种方式, 优化数据通过 PCI 桥的效率。

1. Combining

PCI 桥可以将接收到的多个存储器写总线事务合并为一个突发存储器写总线事务。PCI 桥进行这种 Combining 操作时需要注意数据传送的“顺序”。当 PCI 桥接收到一组物理地址连续的存储器写访问时, 如对 PCI 设备的某段空间的 DW1、DW2 和 DW4 进行存储器写访问时, PCI 桥可以将这组访问转化为一个对 DW1~DW4 的突发存储器写访问, 并使用字节使能信号 C/BE[3:0]#进行控制, 其过程如下所示。

PCI 桥将在数据周期 1 中, 置 C/BE[3:0]#信号为有效表示传递数据 DW1; 在数据周期 2 中, 置 C/BE[3:0]#信号为有效表示传递数据 DW2; 在数据周期 3 中, 置 C/BE[3:0]#信号为无效表示在这个周期中所传递的数据无效, 从而跳过 DW3; 并在数据周期 4 中, 置 C/BE[3:0]#信号为有效表示传递数据 DW4。

目标设备将最终按照发送端的顺序, 接收 DW1、DW2 和 DW4, 采用这种方法在不改变传送序的前提下, 提高了数据的传送效率。值得注意的是, 有些 HOST 主桥也提供这种 Combining 方式, 合并多次数据访问。如果目标设备不支持突发传送方式, 该设备可以使用 Disconnect 周期, 终止突发传送, 此时 PCI 桥/HOST 主桥可以使用多个存储器写总线事务分别传送 DW1、DW2 和 DW4, 而不会影响数据传送。

如果 PCI 桥收到“乱序”的存储器写访问, 如对 PCI 设备的某段空间的 DW4、3 和 1 进行存储器写访问时, PCI 桥不能将这组访问转化为一个对 DW1~4 的突发存储器写访问, 此时 PCI 桥必须使用三个存储器写总线事务转发这些存储器写访问。

2. Merging

PCI 桥可以将收到的多个对同一个 DW 地址的 Byte、Word 进行的存储器写总线事务，合并为一个对这个 DW 地址的存储器写总线事务。PCI 规范并没有要求这些对 Byte、Word 进行的存储器写在一个 DW 的边界之内，但是建议 PCI 桥仅处理这种情况。本节也仅介绍在这种情况下，PCI 桥的处理过程。

PCI 规范允许 PCI 桥进行 Merging 操作的存储器区域必须是可预读的，而可预读的存储器区域必须支持这种 Merging 操作。Merging 操作可以不考虑访问顺序，可以将对 Byte0、Byte1、Byte3 的存储器访问合并为一个 DW，也可以将对 Byte3、Byte1、Byte0 的存储器访问合并为一个 DW。在这种情况下，PCI 总线事务仅需屏蔽与 Byte2 相关的字节使能信号 C/BE2# 即可。

如果 PCI 设备对 Byte1 进行存储器写、然后再对 Byte1、Byte2、Byte3 进行存储器写时，PCI 桥不能将这两组存储器写合并为一次对 DW 进行存储器写操作。但是 PCI 桥可以合并后一组存储器写，即首先对 Byte1 进行存储器写，然后合并后一组存储器写（Byte1、Byte2 和 Byte3）为一个 DW 写，并屏蔽相应的 C/BE0# 信号。Combining 与 Merging 操作之间没有直接联系，PCI 桥可以同时支持这两种方式，也可以支持任何一种方式。

3. Collapsing

Collapsing 指 PCI 桥可以将对同一个地址进行的 Byte、Word 和 DW 存储器写总线事务合并为一个存储器写操作。使用 PCI 桥的 Collapsing 方式是，具有某些条件限制，在多数情况下，PCI 桥不能使用 Collapsing 方式合并多个存储器写总线事务。

当 PCI 桥收到一个对“DW 地址 X”的 Byte3 进行的存储器写总线事务，之后又收到一个对“DW 地址 X”的 Byte、Word 或者 DW 存储器写总线事务，而且后一个对 DW 地址 X 进行的存储器写仍然包含 Byte3 时，如果 PCI 桥支持 Collapsing 方式，就可以将这两个存储器写合并为一个存储器写。

PCI 桥在绝大多数情况下不能支持这种方式，因为很少有 PCI 设备支持这种数据合并方式。通常情况下，对 PCI 设备的同一地址的两次写操作代表不同的含义，因此 PCI 桥不能使用 Collapsing 方式将这两次写操作合并。PCI 规范仅是提出了 Collapsing 方式的概念，几乎没有 PCI 桥支持这种数据合并方式。

3.3 与 Cache 相关的 PCI 总线事务

PCI 总线规范定义了一系列与 Cache 相关的总线事务，以提高 PCI 设备与主存储器进行数据交换的效率，即 DMA 读写的效率。当 PCI 设备使用 DMA 方式向存储器进行读写操作时，一定需要经过 HOST 主桥，而 HOST 主桥通过 FSB 总线^①向存储器控制器进行读写操作时，需要进行 Cache 共享一致性操作。

PCI 设备与主存储器进行的 Cache 共享一致性增加了 HOST 主桥的设计复杂度。在高性能处理器中 Cache 状态机的转换模型十分复杂。而 HOST 主桥是 FSB 上的一个设备，需要按照 FSB 规定的协议处理这个 Cache 一致性，而多级 Cache 的一致性和状态转换模型一直是高

① 在许多处理器中，HOST 主桥与 FSB 之间还存在 SoC 平台总线。

性能处理器设计中的难点。

不同的 HOST 主桥处理 PCI 设备进行的 DMA 操作时，使用的 Cache 一致性的方法并不相同。因为 Cache 一致性操作不仅与 HOST 主桥的设计相关，而且主要与处理器和 Cache Memory 系统设计密切相关。

PowerPC 和 x86 处理器可以对 PCI 设备所访问的存储器进行设置，其设置方法并不相同。其中 PowerPC 处理器，如 MPC8548 处理器，可以使用 Inbound 寄存器的 RTT 字段和 WTT 字段，设置在 PCI 设备进行 DMA 操作时，是否需要进行 Cache 一致性操作，是否可以将数据直接写入 Cache 中。RTT 字段和 WTT 字段的详细说明见第 2.2.3 节。

而 x86 处理器可以使用 MTRR (Memory Type Range Registers) 设置物理存储器区间的属性是否为可 Cache 空间。下文分别讨论在 PowerPC 与 x86 处理器中，PCI 设备进行 DMA 写操作时，如何进行 Cache 一致性操作。

但是与 PowerPC 处理器相比，x86 处理器在处理 PCI 设备的 Cache 一致性上略有不足，特别是网络设备与存储器系统进行数据交换的效率。因为 x86 处理器重点优化的是 PCIe 设备，目前 x86 处理器使用的 IOAT (I/O Acceleration Technology) 技术，显著提高了 PCIe 设备与主存储器进行数据通信的效率。

3.3.1 Cache 一致性的基本概念

PCI 设备对可 Cache 的存储器空间进行 DMA 读写操作的过程较为复杂，有关 Cache Memory 的话题可以独立成书。而不同的处理器系统使用的 Cache Memory 的层次结构和访问机制有较大的差异，这部分内容也是现代处理器系统设计的重中之重。

本节仅介绍在 Cache Memory 系统中与 PCI 设备进行 DMA 操作相关的一些最为基础的概念。在多数处理器系统中，使用了以下概念描述 Cache 一致性的实现过程。

1. Cache 一致性协议

多数 SMP 处理器系统使用了 MESI 协议处理多个处理器之间的 Cache 一致性。该协议也称为 Illinois protocol，在 SMP 处理器系统中得到了广泛的应用。MESI 协议使用四个状态位描述每一个 Cache 行。

- M (Modified) 位。M 位为 1 时表示当前 Cache 行中包含的数据与存储器中的数据不一致，而且它仅在本 CPU 的 Cache 中有效，不在其他 CPU 的 Cache 中存在副本，在这个 Cache 行的数据是当前处理器系统中最新的数据副本。当 CPU 对这个 Cache 行进行替换操作时，必然会引发系统总线的写周期，将 Cache 行中数据与内存中的数据同步。
- E (Exclusive) 位。E 位为 1 时表示当前 Cache 行中包含的数据有效，而且该数据仅在当前 CPU 的 Cache 中有效，而不在其他 CPU 的 Cache 中存在副本。在该 Cache 行中的数据是当前处理器系统中最新的数据副本，而且与存储器中的数据一致。
- S (Shared) 位。S 位为 1 表示 Cache 行中包含的数据有效，而且在当前 CPU 和至少其他一个 CPU 中具有副本。在该 Cache 行中的数据是当前处理器系统中最新的数据副本，而且与存储器中的数据一致。
- I (Invalid) 位。I 位为 1 表示当前 Cache 行中没有有效数据或者该 Cache 行没有使能。MESI 协议在进行 Cache 行替换时，将优先使用 I 位为 1 的 Cache 行。

MESI 协议还存在一些变种，如 MOESI 协议和 MESIF 协议。基于 MOESI 协议的 Cache 一致性模型如图 3-5 所示。AMD 处理器就使用 MOESI 协议。不同的处理器在实现 MOESI 协议时，状态机的转换原理类似，但是在处理上仍有较大的区别。

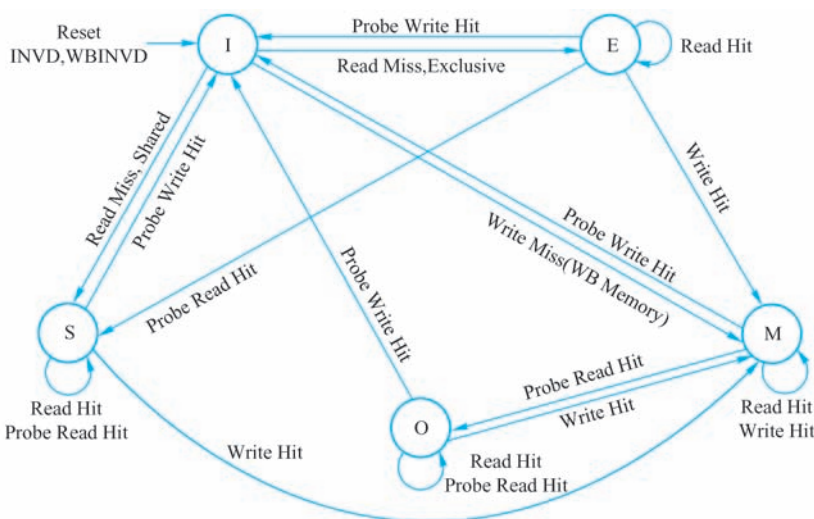


图 3-5 基于 MOESI 协议的 Cache 一致性模型

MOESI 协议引入了一个 O (Owned) 状态，并在 MESI 协议的基础上，重新定义了 S 状态，而 E、M 和 I 状态和 MESI 协议的对应状态相同。

- O 位。O 位为 1 表示在当前 Cache 行中包含的数据是当前处理器系统最新的数据副本，而且在其他 CPU 中一定具有该 Cache 行的副本，其他 CPU 的 Cache 行状态为 S。如果主存储器的数据在多个 CPU 的 Cache 中都具有副本时，有且仅有一个 CPU 的 Cache 行状态为 O，其他 CPU 的 Cache 行状态只能为 S。与 MESI 协议中的 S 状态不同，状态为 O 的 Cache 行中的数据与存储器中的数据并不一致。
- S 位。在 MOESI 协议中，S 状态的定义发生了细微的变化。当一个 Cache 行状态为 S 时，其包含的数据并不一定与存储器一致。如果在其他 CPU 的 Cache 中不存在状态为 O 的副本时，该 Cache 行中的数据与存储器一致；如果在其他 CPU 的 Cache 中存在状态为 O 的副本时，Cache 行中的数据与存储器不一致。

在一个处理器系统中，主设备（CPU 或者外部设备）进行存储器访问时，将试图从存储器系统（主存储器或者其他 CPU 的 Cache）中获得最新的数据副本。如果该主设备访问的数据没有在本地命中时，将从其他 CPU 的 Cache 中获取数据，如果这些数据仍然没有在其他 CPU 的 Cache 中命中，主存储器将提供数据。外设设备进行存储器访问时，也需要进行 Cache 共享一致性。

在 MOESI 模型中，“Probe Read”表示主设备从其他 CPU 中获取数据副本的目的是为了读取数据；而“Probe Write”表示主设备从其他 CPU 中获取数据副本的目的是为了写入数据；“Read Hit”和“Write Hit”表示主设备在本地 Cache 中获得数据副本；“Read Miss”和“Write Miss”表示主设备没有在本地 Cache 中获得数据副本；“Probe Read Hit”和“Probe Write Hit”表示主设备在其他 CPU 的 Cache 中获得数据副本。

本节为简便起见，仅介绍 CPU 进行存储器写和与 O 状态相关的 Cache 行状态迁移，CPU 进行存储器读的情况相对较为简单，请读者自行分析这个过程。

当 CPU 对一段存储器进行写操作时，如果这些数据在本地 Cache 中命中时，其状态可能为 E、S、M 或者 O。

- 状态为 E 或者 M 时，数据将直接写入 Cache 中，并将状态改为 M。
- 状态为 S 时，数据将直接写入 Cache 中，并将状态改为 M，同时其他 CPU 保存该数据副本的 Cache 行状态将从 S 或者 O 迁移到 I (Probe Write Hit)。
- 状态为 O 时，数据将直接写入 Cache 中，并将状态改为 M，同时其他 CPU 保存该数据副本的 Cache 行状态将从 S 迁移到 I (Probe Write Hit)。

当 CPU A 对一段存储器进行写操作时，如果这些数据没有在本本地 Cache 中命中时，而在其他 CPU，如 CPU B 的 Cache 中命中时，其状态可能为 E、S、M 或者 O。其中 CPU A 使用 CPU B 在同一个 Cache 共享域中。

- Cache 行状态为 E 时，CPU B 将该 Cache 行状态改为 I；而 CPU A 将从本地申请一个新的 Cache 行，将数据写入，并将该 Cache 行状态更新为 M。
- Cache 行状态为 S 时，CPU B 将该 Cache 行状态改为 I，而且具有同样副本的其他 CPU 的 Cache 行也需要将状态改为 I；而 CPU A 将从本地申请一个 Cache 行，将数据写入，并将该 Cache 行状态更新为 M。
- Cache 行状态为 M 时，CPU B 将原 Cache 行中的数据回写到主存储器，并将该 Cache 行状态改为 I；而 CPU A 将从本地申请一个 Cache 行，将数据写入，并将该 Cache 行状态更新为 M。
- Cache 行状态为 O 时，CPU B 将原 Cache 行中的数据回写到主存储器，并将该 Cache 行状态改为 I，具有同样数据副本的其他 CPU 的 Cache 行也需要将状态从 S 更改为 I；CPU A 将从本地申请一个 Cache 行，将数据写入，并将该 Cache 行状态更新为 M。

Cache 行状态可以从 M 迁移到 O。例如当 CPU A 读取的数据从 CPU B 中命中时，如果在 CPU B 中 Cache 行的状态为 M 时，将迁移到 O，同时 CPU B 将数据传送给 CPU A 新申请的 Cache 行中，而且 CPU A 的 Cache 行状态将被更改为 S。

当 CPU 读取的数据在本地 Cache 中命中，而且 Cache 行状态为 O 时，数据将从本地 Cache 获得，并不会改变 Cache 行状态。如果 CPU A 读取的数据在其他 Cache 中命中，如在 CPU B 的 Cache 中命中而且其状态为 O 时，CPU B 将该 Cache 行状态保持为 O，同时 CPU B 将数据传送给 CPU A 新申请的 Cache 行中，而且 CPU A 的 Cache 行状态将被更改为 S。

在某些应用场合，使用 MOESI 协议将极大提高 Cache 的利用率，因为该协议引入了 O 状态，从而在发送 Read Hit 的情况时，不必将状态为 M 的 Cache 回写到主存储器，而是直接从一个 CPU 的 Cache 将数据传递到另外一个 CPU。目前 MOESI 协议在 AMD 和 RMI 公司的处理器中得到了广泛的应用。

Intel 提出了另外一种 MESI 协议的变种，即 MESIF 协议，该协议与 MOESI 协议有较大的不同，也远比 MOESI 协议复杂，该协议由 Intel 的 QPI (Quick Path Interconnect) 技术引入，其主要目的是解决“基于点到点的全互连处理器系统”的 Cache 共享一致性问题，而不是“基于共享总线的处理器系统”的 Cache 共享一致性问题。

在基于点到点互连的 NUMA（Non-Uniform Memory Architecture）处理器系统中，包含多个子处理器系统，这些子处理器系统由多个 CPU 组成。如果这个处理器系统需要进行全机 Cache 共享一致性，该处理器系统也被称为 ccNUMA（Cache Coherent NUMA）处理器系统。MESIF 协议主要解决 ccNUMA 处理器结构的 Cache 共享一致性问题，这种结构通常使用目录表，而不使用总线监听处理 Cache 的共享一致性。

MESIF 协议引入了一个 F（Forward）状态。在 ccNUMA 处理器系统中，可能在多个处理器的 Cache 中存在相同的数据副本，在这些数据副本中，只有一个 Cache 行的状态为 F，其他 Cache 行的状态都为 S。Cache 行的状态位为 F 时，Cache 中的数据与存储器一致。

当一个数据请求方读取这个数据副本时，只有状态为 F 的 Cache 行，可以将数据副本转发给数据请求方，而状态位为 S 的 Cache 不能转发数据副本。从而 MESIF 协议有效解决了在 ccNUMA 处理器结构中，所有状态位为 S 的 Cache 同时转发数据副本给数据请求方，而造成的数据拥塞。

在 ccNUMA 处理器系统中，如果状态位为 F 的数据副本被其他 CPU 复制，F 状态位将会被迁移，新建的数据副本的状态位将为 F，而老的数据副本的状态位将改变为 S。当状态位为 F 的 Cache 行被改写后，ccNUMA 处理器系统需要首先 Invalidate 状态位为 S 其他的 Cache 行，之后将 Cache 行的状态更新为 M。

独立地研究 MESIF 协议并没有太大意义，该协议由 Boxboro-EX 处理器系统[⊖]引入，目前 Intel 并没有公开 Boxboro-EX 处理器系统的详细设计文档。MESIF 协议仅是解决该处理器系统中 Cache 一致性的一个功能，该功能的详细实现与 QPI 的 Protocol Layer 相关，QPI 由多个层次组成，而 Protocol Layer 是 QPI 的最高层。

对 MESIF 协议 QPI 互连技术有兴趣的读者，可以在深入理解“基于目录表的 Cache 一致性协议”的基础上，阅读 Robert A. Maddox, Gurbir Singh and Robert J. Safranek 合著的书籍“*Weaving High Performance Multiprocessor Fabric*”以了解该协议的实现过程和与 QPI 互连技术相关的背景知识。

值得注意的是，MESIF 协议解决主要的问题是 ccNUMA 架构中 SMP 子系统与 SMP 子系统之间 Cache 一致性。而在 SMP 处理器系统中，依然需要使用传统的 MESI 协议。Nehalem EX 处理器也可以使用 MOESI 协议进一步优化 SMP 系统使用的 Cache 一致性协议，但是并没有使用该协议。

为简化起见，本章假设处理器系统使用 MESI 协议进行 Cache 共享一致性，而不是 MOESI 协议或者 MESIF 协议。

2. HIT#和 HITM#信号

在 SMP 处理器系统中，每一个 CPU 都使用 HIT#和 HITM#信号反映 HOST 主桥访问的地址是否在自己的 Cache 中命中。当 HOST 主桥访问存储器时，CPU 将驱动 HITM#和 HIT#信号，其描述如表 3-1 所示。

⊖ Boxboro-EX 处理器系统由多个 Nehalem EX 处理器组成，而 Nehalem EX 处理器由两个 SMP 处理器系统组成，一个 SMP 处理器系统由 4 个 CPU 组成，而每一个 CPU 具有 2 个线程。其中 SMP 处理器系统之间使用 QPI 进行连接，而在一个 SMP 处理器内部的各个 CPU 仍然使用 FSB 连接。

表 3-1 HITM#和 HIT#信号的含义

HITM#	HIT#	描 述
1	1	表示 HOST 主桥访问的地址没有在 CPU 的 Cache 中命中
1	0	表示 HOST 主桥访问的地址在 CPU 的 Cache 中命中，而且 Cache 的状态为 S (Shared) 或者 E (Exclusive)，即 Cache 中的数据与存储器的数据一致
0	1	表示 HOST 主桥访问的地址在 CPU 的 Cache 中命中，而且 Cache 的状态为 M (Modified)，即 Cache 中的数据与存储器的数据不一致，在 Cache 中保存最新的数据副本
0	0	MESI 协议规定这种情况不允许出现，但是在有些处理器系统中仍然使用了这种状态，表示暂时没有获得是否在 Cache 中命中的信息，需要等待几拍后重试

HIT#和 HITM#信号是 FSB 中非常重要的两个信号，各个 CPU 的 HIT#和 HITM#信号通过“线与”方式直接相连^①。而在一个实际 FSB 中，还包括许多信号，本节并不会详细介绍这些信号。

3. Cache 一致性协议中使用的 Agent

在处理器系统中，与 Cache 一致性相关的 Agent 如下所示。

- Request Agent。FSB 总线事务的发起设备。在本节中，Request Agent 特指 HOST 主桥。实际上在 FSB 总线上的其他设备也可以成为 Request Agent，但这些 Request Agent 并不是本节的研究重点。Request Agent 需要进行总线仲裁后，才能使用 FSB，在多数处理器的 FSB 中，需要对地址总线与数据总线分别进行仲裁。
- Snoop Agents。FSB 总线事务的监听设备。Snoop Agents 为 CPU，在一个 SMP 处理器系统中，有多个 CPU 共享同一个 FSB，此时这些 CPU 都是这条 FSB 上的 Snoop Agents。Snoop Agents 监听 FSB 上的存储器读写事务，并判断这些总线事务访问的地址是否在 Cache 中命中。Snoop Agents 通过 HIT#和 HITM#信号向 FSB 通知 Cache 命中的结果。在某些情况下，Snoop Agents 需要将 Cache 中的数据回写到存储器，同时为 Request Agent 提供数据。
- Response Agent。FSB 总线事务的目标设备。在本节中，Response Agent 特指存储器控制器。Response Agent 根据 Snoop Agents 提供的监听结果，决定如何接收数据或者向 Request Agent 设备提供数据。在多数情况下，当前数据访问没有在 Snoop Agents 中命中时，Response Agent 需要提供数据，此外 Snoop Agents 有时需要将数据回写到 Response Agent 中。

4. FSB 的总线事务

一个 FSB 的总线事务由多个阶段组成，包括 Request Phase、Snoop Phase、Response Phase 和 Data Phase。目前在多数高端处理器中，FSB 支持流水操作，即在同一个时间段内，不同的阶段可以重叠，如图 3-6 所示。

在一个实际的 FSB 中，一个总线事务还可能包含 Arbitration Phase 和 Error Phase。而本节仅讲述图 3-6 中所示的 4 个基本阶段。

- Request Phase。Request Agent 在获得 FSB 的地址总线的使用权后，在该阶段将访问数据区域的地址和总线事务类型发送到 FSB 上。

^① HIT#和 HITM#信号是 Open Drain (开漏) 信号，Open Drain 信号可以直接相连，而不用使用逻辑门。

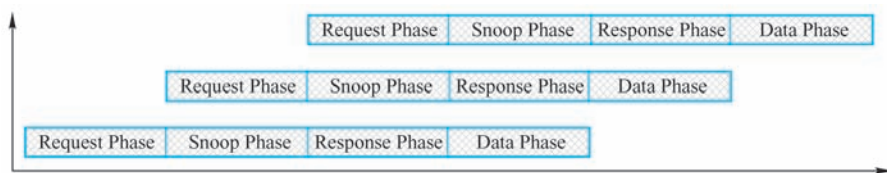


图 3-6 FSB 的流水操作

- Snoop Phase。Snoop Agents 根据访问数据区域在 Cache 中的命中情况，使用 HIT#和 HITM#信号，向其他 Agents 通知 Cache 一致性的结果。有时 Snoop Agent 需要将数据回写到存储器。
- Reponse Phase。Response Agent 根据 Request 和 Snoop Phase 提供的信号，可以要求 Request Agent 重试 (Retry)，或者 Response Agent 延时处理 (Defer) 当前总线事务。在 FSB 总线事务的各个阶段中，该步骤的处理过程最为复杂。本章将在下文结合 PCI 设备的 DMA 读写执行过程，说明该阶段的实现原理。
- Data Phase。一些不传递数据的 FSB 总线事务不包含该阶段。该阶段用来进行数据传递，包括 Request Agent 向 Response Agent 写入数据；Response Agent 为 Request Agent 提供数据；和 Snoop Agent 将数据回写到 Response Agent。

下面将使用本小节中的概念，描述在 PCI 总线中，与 Cache 相关的总线事务，并讲述相关的 FSB 的操作流程。

3.3.2 PCI 设备对不可 Cache 的存储器空间进行 DMA 读写

在 x86 处理器和 PowerPC 处理器中，PCI 设备对“不可 Cache 的存储器空间”进行 DMA 读写的过程并不相同。其中 PowerPC 处理器对“不可 Cache 的存储器空间”进行 DMA 读写进行了专门的处理，而 x86 处理器在对这类空间操作时，效率相对较低。

1. x86 处理器

x86 处理器使用 MTRR (Memory Type Range Register) 寄存器设置存储器空间的属性，如果存储器空间为“可 Cache 空间”，x86 处理器还可以进一步设置这段空间为“Write Through”“Write Combining”“Write Protect”和“Write Back”。但是这些设置与 PCI 设备进行 DMA 操作时，是否进行 Cache 一致性操作并没有直接关系。

在 x86 处理器系统中，一个 PCI 设备进行 DMA 写操作，可以将数据从 PCI 设备写入到主存储器中。这个数据首先需要通过 HOST 主桥，然后经过 FSB 发送到存储器控制器。虽然在 x86 处理器系统中，CPU 知道这个存储器区域是否为“可 Cache 的”，但是 HOST 主桥并不知道 PCI 设备访问的存储器地址是否为“可 Cache 的”，因此都需要使用“Cache 一致”的 FSB 总线传送事务^①进行存储器写操作，从而数据在发向 FSB 时，CPU 必须要进行总线监听，通知 FSB 总线这段空间是“不可 Cache 的”。

在 x86 处理器中，PCI 设备向不可 Cache 的存储器空间进行读操作时，CPU 也必须进行

① FSB 总线定义了许多总线事务，有的 FSB 总线提供了一个 Snoop 信号，该信号为 1 时表示当前 FSB 的总线事务需要进行 Cache 共享一致性，为 0 时不需要进行 Cache 共享一致性。

Cache 共享一致性操作，而这种没有必要的 Cache 共享一致性操作将影响 PCI 总线的传送效率。当 PCI 设备所访问的存储器空间没有在 CPU 的 Cache 命中时，CPU 会通知 FSB，数据没有在 Cache 中命中，此时 PCI 设备访问的数据将从存储器中直接读出。

x86 处理器在前端总线上进行 Cache 共享一致性操作时，需要使用 Snoop Phase，如果 PCI 设备能事先得知所访问的存储器是“不可 Cache 的”，就不必在前端总线上进行 Cache 共享一致性操作，即 FSB 总线事务不必包含 Snoop Phase，从而可以提高前端总线的使用效率。但是 x86 处理器并不支持这种方式。

在 x86 处理器系统中，无论 PCI 设备访问的存储器空间是否为“不可 Cache 的”，都需要进行 Cache 共享一致性操作。这也是 PCI 总线在 x86 处理器使用中的一个问题。而 PCIe 总线通过在数据报文中设置“Snooping”位解决了这个问题，有关 PCIe 总线 Snooping 位的内容参见第 6.1.3 节。

2. PowerPC 处理器

在 MPC8548 处理器中，HOST 主桥可以通过 PIWARn 寄存器^①的 RTT 字段和 WTT 字段预知 PCI 设备访问的存储器空间是否为可 Cache 空间。当 HOST 主桥访问“不可 Cache 空间”时，可以使用 FSB 总线的“不进行 Cache 一致性”的总线事务。

此时 PowerPC 处理器不会在 FSB 总线中进行 Cache 一致性操作，即忽略 FSB 总线事务的 Snoop Phase。PCI 设备进行 DMA 写时，数据将直接进入主存储器，而 PCI 设备进行 DMA 读所读取的数据将直接从主存储器获得。与 x86 处理器相比，PowerPC 处理器可以忽略 CPU 进行总线监听的动作，从而提高了 FSB 传送效率。

3.3.3 PCI 设备对可 Cache 的存储器空间进行 DMA 读写

PCI 设备向“可 Cache 的存储器空间”进行读操作的过程相对简单。对于 x86 处理器或者 PowerPC 处理器，如果访问的数据在 Cache 中命中，CPU 会通知 FSB 总线，PCI 设备所访问的数据在 Cache 中。

首先 HOST 主桥发起存储器读总线事务，并在 Request Phase 中提供地址。Snoop Agent 在 Snoop Phase 进行总线监听，并通过 HIT#和 HITM#信号将监听结果通知给 Response Agent。如果 Cache 行的状态为 E 时，Response Agent 将提供数据，而 CPU 不必改变 Cache 行状态。如果 Snoop Agent 可以直接将数据提供给 HOST 主桥，无疑数据访问的延时更短，但是采用这种方法无疑会极大地提高 Cache Memory 系统的设计难度，因此采用这种数据传送方式的处理器^②并不多。

如果 Cache 行的状态为 M 时，Response Agent 在 Response Phase 阶段，要求 Snoop Agent 将 Cache 中的数据回写到存储器，并将 Cache 行状态更改为 E。Snoop Agent 在 Data Phase 将 Cache 中的数据回写给存储器控制器，同时为 HOST 主桥提供数据。Snoop Agent 也可以直接将数据提供给 HOST 主桥，不需要进行数据回写过程，也不更改 Cache 行状态，但是采用这种方法会提高 Cache Memory 系统的设计难度。

① 该寄存器在 Inbound 寄存器组中，详见第 2.2.3 节。

② 目前 Cortex A8/A9 和 Intel 的 Sandy Bridge 处理器支持这种方式。

如果 PCI 设备访问的数据没有在 Cache 中命中，Snoop Agents 会通知 FSB 总线，PCI 设备所访问的数据不在 Cache 中，此时存储器控制器（Response Agent）将在 Data Phase 向 HOST 主桥提供数据。

PCI 设备向“可 Cache 的”存储器区域进行写操作，无论对于 PowerPC 处理器还是 x86 处理器，都较为复杂。当 HOST 主桥通过 FSB 将数据发送给存储器控制器时，在这个系统总线上的所有 CPU 都需要对这个 PCI 写操作进行监听，并根据监听结果，合理地改动 Cache 行状态，并将数据写入存储器。

下面以图 3-7 所示的 SMP 处理器系统为例，说明 PCI 设备对“可 Cache 的存储器空间”进行 DMA 写的实现过程。

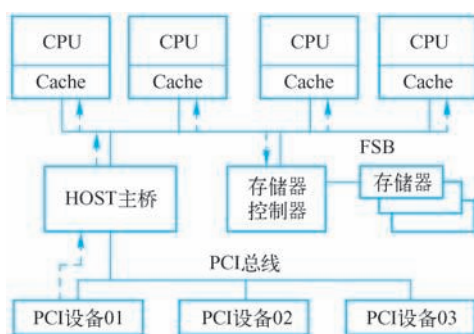


图 3-7 PCI 设备向可 Cache 的存储器空间进行写操作

在图 3-7 所示的处理器系统中，存在 4 个 CPU，这 4 个 CPU 通过一条 FSB 连接在一起，CPU 之间使用 MESI 协议进行 Cache 一致性处理，而 HOST 主桥和存储器控制器与 FSB 直接相连。HOST 主桥向存储器控制器传递数据时，需要处理 Cache 的一致性。

在这个处理器系统中，当 PCI 设备，如 PCI 设备 01，进行 DMA 写操作时，数据将首先到达 HOST 主桥，而 HOST 主桥将首先接管该 PCI 设备数据访问并将其转换为 FSB 总线事务，并在 Request Phase 中，提供本次 FSB 总线事务的地址。CPU 将在 Snoop Phase 对这个地址进行监听，判断当前地址在 Cache 中的命中情况。

当 HOST 主桥访问的地址不在 Cache 中命中时，此时在处理器系统中，所有 CPU 都没有驱动 HIT#和 HITM#信号，HIT#和 HITM#信号都为 1，表示 HOST 主桥访问的地址没有在 CPU 的 Cache 中命中，HOST 主桥可以简单地将数据写入存储器。当 HOST 主桥访问的存储器地址在 Cache 中命中时，Cache 行的状态可以为 S、E 或者为 M，此时处理器系统的处理过程相对较为复杂，下一节将专门讨论这种情况。

3.3.4 PCI 设备进行 DMA 写时发生 Cache 命中

如果 PCI 设备访问的地址在某个 CPU 的 Cache 行中命中时，可能会出现三种情况。

第一种情况是命中的 Cache 行其状态为 E，即 Cache 行中的数据与存储器中的数据一致；而第二种情况是命中的 Cache 行其状态为 S。其中 E 位为 1 表示该数据在 SMP 处理器系统中，有且仅有一个 CPU 的 Cache 中具有数据副本；而 S 位为 1 表示在 SMP 处理器系统中，该数据至少在两个以上 CPU 的 Cache 中具有数据副本。

当 Cache 行状态为 E 时，这种情况比较容易处理。因为 PCI 设备（通过 HOST 主桥）写

入存储器的信息比 Cache 行中的数据新，而且 PCI 设备在进行 DMA 写操作之前，存储器与 Cache 中数据一致，此时 CPU 仅需要在 Snoop Phase 使无效（Invalidate）这个 Cache 行，然后 FSB 总线事务将数据写入存储器即可。当然如果 FSB 总线事务可以将数据直接写入 Cache，并将 Cache 行的状态更改为 M，也可提高 DMA 写的效率，这种方式的实现难度较大，第 3.3.5 节将介绍这种优化方式。

Cache 行状态为 S 时的处理情况与状态为 E 时的处理情况大同小异，PCI 设备在进行写操作时也将数据直接写入主存储器，并使无效状态为 S 的 Cache 行。

第三种情况是命中的 Cache 行其状态为 M（Modified），即 Cache 行中的数据与存储器的数据不一致，Cache 行中保存最新的数据副本，主存储器中的部分数据无效。对于 SMP 系统，此时有且仅有一个 CPU 中的 Cache 行的状态为 M，因为 MESI 协议规定存储器中的数据不能在多个 CPU 的 Cache 行中的状态为 M。

我们假定一个处理器的 Cache 行长度为 32B，即 256 bit。当这个 Cache 行的状态为 M 时，表示这个 Cache 行的某个字节、双字、几个双字，或者整个 Cache 行中的数据比主存储器中含有的数据新。

假设 HOST 主桥访问的地址，在 Snoop Phase，通过 CPU 进行总线监听后，发现其对应的 Cache 行状态为 M。此时 HOST 主桥进行存储器写操作时，处理情况较为复杂，此时这些状态为 M 的数据需要回写到主存储器。

下面考虑如图 3-8 所示的实例。假定处理器的 Cache 使用回写（Write-Back）策略进行更新。在这个实例中，HOST 主桥对存储器的某个地址进行写操作，而所有 CPU 通过 FSB 总线进行总线监听时发现，HOST 主桥使用的这个目的地址在某个 CPU 的 Cache 行命中，此时这个 CPU 将置 HITM#信号为 0，并置 HIT#信号为 1，表示当前 Cache 行中含有的数据比存储器中含有的数据更新。

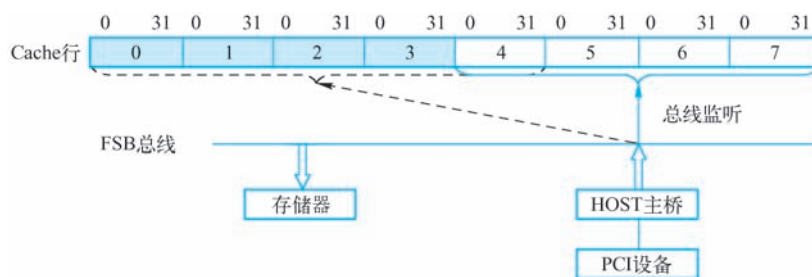


图 3-8 PCI 设备向 Cache 行状态为 M 的存储器进行写操作

假设此时在 Cache 行中，阴影部分的数据比存储器中的数据新，而其他数据与存储器保持一致，即在这个 Cache 行中第 0~3 个双字的数据是当前处理器系统中最新的数据，而第 4~7 个双字中的数据与存储器保持一致。

如果 PCI 设备向存储器写的数据区域可以完全覆盖这些阴影部分，如对第 0~5 个双字进行写操作时，这种情况不难处理。此时 CPU 只需在总线监听阶段，将这个 Cache 行使无效，然后将数据写入存储器即可。因为完成这个存储器写操作之后，PCI 设备写入的数据是最新的，而且这个最新的数据将完全覆盖在 Cache 行中阴影部分的数据，所以 CPU 只需要简单地将这个 Cache 行使无效即可。

然而 PCI 设备 (HOST 主桥) 无法预先知道这些 Cache 行中的数据哪些是有效的, 哪些是无效的, 而仅知道命中了一个“被修改过”的 Cache 行, 从而 PCI 设备 (HOST 主桥) 无法保证对 Cache 行中有效数据进行覆盖。因此 PCI 设备对存储器进行写操作时, 不能简单地使无效 (Invalid) 状态位为 M 的 Cache 行。

仍然以图 3-8 为例, 考虑一个 PCI 设备将 4 个双字 (第 4~7 个双字) 的数据写入一个存储器中, 这 4 个双字所访问的数据在某个 CPU 的 Cache 行中命中, 该 Cache 行的状态为 M, 而且这个 Cache 行的前 4 个双字曾被处理器修改过。

此时 CPU 对 FSB 总线监听时, 不能简单将当前 Cache 行使无效, 因为这个使无效操作将丢失阴影部分的有效数据。这个阴影部分中的有效数据并没有被 PCI 设备重新写入, 因此在整个处理器系统中, 这个阴影部分仍然包含最新的数据。将最新的数据丢弃显然是一种错误做法, 会导致处理器系统的崩溃。

为此 HOST 主桥需要专门处理这种情况, 不同的 HOST 主桥采用了不同的方法处理这种情况, 但无外乎以下三种方法。

1) CPU 进行总线监听后发现, HOST 主桥访问的数据命中了一个状态位为 M 的 Cache 行, 此时存储器控制器将通知 HOST 主桥重试或者延时处理, 并暂时停止 HOST 主桥发起的这次存储器写操作。随后 CPU 将状态位为 M 的 Cache 行与存储器进行同步后, 再使无效这个 Cache 行。之后 HOST 主桥在合适的时机, 重新发起被 HOST 主桥要求重试的总线事务, 此时 CPU 再次进行总线监听时不会再次出现 Cache 命中的情况, 因此 HOST 主桥可以直接将数据写入存储器。许多 HOST 主桥使用这种方法处理 PCI 设备的存储器写总线事务。

2) 首先 HOST 主桥将接收 PCI 设备进行 DMA 写的的数据, 并将这些数据放入存储器控制器的一个缓冲区中, 同时结束 PCI 设备的存储器写总线事务。之后 CPU 进行总线监听, 如果 CPU 发现 HOST 主桥访问的数据命中了一个状态位为 M 的 Cache 行时, 则这个 Cache 行放入存储器控制器的另一个缓冲区后, 使无效这个 Cache 行。最后存储器控制器将这两个缓冲区的数据合并然后统一写入存储器中。

3) HOST 主桥并不结束当前 PCI 总线周期, 而直接进行总线监听, 如果 CPU 进行总线监听发现 HOST 主桥访问的数据命中了一个状态位为 M 的 Cache 行时, 则将这个 Cache 行整体写入存储器控制器的缓冲区后使无效这个 Cache 行, 之后 HOST 主桥开始从 PCI 设备接收数据, 并将这些数据直接写入这个缓冲区中。最后 HOST 主桥结束 PCI 设备的存储器写总线周期, 同时存储器控制器将这个缓冲区内的数据写入存储器。

以上这几种情况是 PCI 设备进行存储器写时, HOST 主桥可能的处理情况, 其中第 1 种方法最常用。而 x86 处理器使用的 implicit writeback 方式, 与第 2 种方法基本类似。第 3 种方法与第 2 种方法并没有本质不同。

但是如果 PCI 设备对一个或者多个完整 Cache 行的存储器区域进行写操作时, 上述过程显得多余。对完整 Cache 行进行写操作, 可以保证将 Cache 行对应的存储器区域完全覆盖, 此时 Cache 行中的数据在 PCI 设备完成这样的操作后, 在处理器系统中将不再是最新的。PCI 设备进行这样的存储器写操作时, 可以直接将数据写入存储器, 同时直接使无效状态为 M 的 Cache 行。

PCI 总线使用存储器写并无效 (Memory Write and Invalidate) 总线事务, 支持这种对一个完整 Cache 行进行的存储器写总线事务。PCI 设备使用这种总线事务时, 必须事先知道当

前处理器系统中 CPU 使用的 Cache 行大小，使用这种总线事务时，一次总线事务传递数据的大小必须以 Cache 行为单位对界。为此 PCI 设备必须使用配置寄存器 Cache Line Size 保存当前 Cache 行的大小，Cache Line Size 寄存器在 PCI 配置空间的位置见图 2-9。

存储器读（Memory Read）、存储器多行读（Memory Read Multiple）和存储器单行读（Memory Read Line）总线事务也是 PCI 总线中的重要总线事务，这些总线事务不仅和 Cache 有关，还和 PCI 总线的预读机制有关，在第 3.4.5 节中将重点介绍这些总线事务。

3.3.5 DMA 写时发生 Cache 命中的优化

在许多高性能处理器中，还提出了一些新的概念，以加速外设到存储器的 DMA 写过程。如 Freescale 的 I/O Stashing 和 Intel 的 IOAT 技术。

如图 3-8 所示，当设备进行存储器写时，如果可以对 Cache 直接进行写操作，即便这个存储器写命中了一个状态为 M 的 Cache 行，也不必将该 Cache 行的数据回写到存储器中，而是直接将数据写入 Cache，之后该 Cache 行的状态依然为 M。采用这种方法可以有效提高设备对存储器进行写操作的效率。采用直接向 Cache 行写的方法，PCI 设备对存储器写命中一个状态为 M 的 Cache 行时，将执行以下操作。

1) HOST 主桥将对存储器的写请求发送到 FSB 总线上。

2) CPU 通过对 FSB 监听，发现该写请求在某个 Cache 行中命中，而且该 Cache 行的状态为 M。

3) HOST 主桥将数据直接写入 Cache 行中，并保持 Cache 行的状态为 M。注意此时设备不需要将数据写入存储器中。

从原理上看，这种方法并没有奇特之处，只要 Cache 能够提供一个接口，使外部设备能够直接写入即可。但是从具体实现上看，设备直接将数据写入 Cache 中，还是有相当大的难度。特别是考虑在一个处理器中，可能存在多级 Cache，当 CPU 进行总线监听时，可能是在 L1、L2 或者 L3 Cache 中命中，此时的情况较为复杂，多级 Cache 间的协议状态机远比 FSB 总线协议复杂得多。

在一个处理器系统中，如果 FSB 总线事务在“与 FSB 直接相连的 Cache”中命中时，这种情况相对容易处理；但是在与 BSB（Back-Side Bus）直接相连的 Cache 命中时，这种情况较难处理。下面分别对这两种情况进行讨论，在一个处理器中，采用 FSB 和 BSB 连接 Cache 的拓扑如图 3-9 所示。

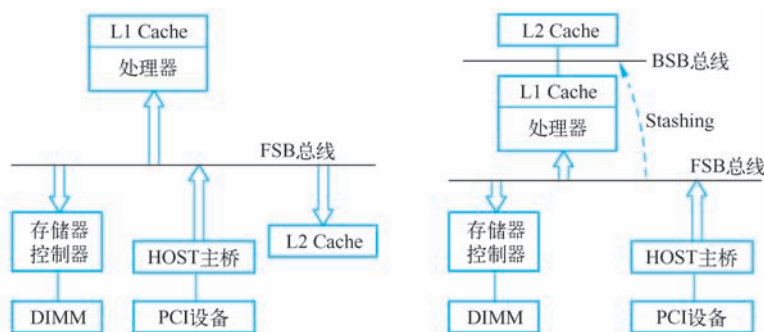


图 3-9 采用 FSB/BSB 进行 Cache 连接

当采用 FSB 总线连接 L2 Cache 时, L2 Cache 直接连接到 FSB 总线上, 设备通过 FSB 总线向 L2 Cache 进行写操作并不难实现, MPC8548 处理器就是采用了这种结构将 L2 Cache 直接连接到 FSB 总线上。

但是由于 FSB 总线的频率低于 BSB 总线频率, 因此采用这种结构将影响 L2 Cache 的访问速度, 为此高端处理器多采用 BSB 总线连接 L2 Cache, x86 处理器在 Pentium Pro 之后的高性能处理器都使用 BSB 总线连接 L2 Cache, Freescale 的 G4 系列处理器和最新的 P4080 处理器也使用 BSB 总线连接 L2 Cache。

当 L2 Cache 没有直接连接到 FSB 上时, 来自外部设备的数据并不容易到达 BSB 总线。除了需要考虑 Cache 连接在 BSB 总线的情况外, 在外部设备进行 DMA 操作时, 还需要考虑多处理器系统的 Cache 共享一致性协议。设计一个专用通道, 将数据从外部设备直接写入处理器的 Cache 中并不容易实现。Intel 的 IOAT 和 Freescale 的 I/O Stashing 可能使用了这种专用通道技术, 直接对 L1 和 L2 Cache 进行写操作, 并在极大增加了设计复杂度的前提下, 提高了处理器系统的整体效率。

以上对 Cache 进行直接写操作, 仅是 Intel 的 IOAT 和 Freescale 的 I/O Stashing 技术的一个子集。目前 Intel 和 Freescale 没有公开这些技术的具体实现细节。在一个处理器系统中, 可能存在多级 Cache, 这些 Cache 的层次组成结构和状态机模型异常复杂, 本章对这些内容不做进一步说明。

3.4 预读机制

随着处理器制造工艺的进步, 处理器主频越来越高, 存储器和外部设备的访问速度虽然也得到极大的提升, 但是依然不与处理器主频的提升速度成正比, 从而处理器的运行速度和外部设备的访问速度之间的差距越来越大, 存储器瓶颈问题愈发严重。虽然 Cache 的使用有效缓解了存储器瓶颈问题, 但是仅使用 Cache 远远不够。

因为无论 Cache 的命中率有多高, 总有发生 Cache 行 Miss 的情况。一旦 Cache 行出现 Miss, 处理器必须启动存储器周期, 将需要的数据从存储器重新填入 Cache 中, 这在某种程度上增加了存储器访问的开销。

使用预读机制可以在一定程度上降低 Cache 行失效所带来的影响。处理器系统可以使用的预读机制, 包括指令 Fetch、数据预读、外部设备的预读队列和操作系统提供的预读策略。本章将简要介绍指令 Fetch, 并重点介绍 CPU 如何对主存储器和外部设备进行数据预读。并以此为基础, 详细说明 PCI 总线使用的预读机制。

3.4.1 指令 Fetch

指令预读是 CPU 指令流水的一个阶段。

在一段程序中, 存在大量的分支预测指令, 因而在某种程度上增加了指令 Fetch 的难度。因此如何判断程序的执行路径是指令流水首先需要解决的问题。

在 CPU 中通常设置了分支预测单元 (Branch Predictor), 在分支指令执行之前, 分支预测单元需要预判分支指令的执行路径, 从而进行指令 Fetch。但是分支预测单元并不会每次

都能正确判断分支指令的执行路径，这为指令 Fetch 制造了不小的麻烦，在这个背景下许多分支预测策略应运而生。

这些分支预测策略主要分为静态预测和动态预测两种方法。静态预测方法的主要实现原理是利用 Profiling 工具，静态分析程序的架构，并为编译器提供一些反馈信息，从而对程序的分支指令进行优化。如在 PowerPC 处理器的转移指令中存在一个“at”字段，该字段可以向 CPU 提供该转移指令是否 Taken[⊖]的静态信息。在 PowerPC 处理器中，条件转移指令“bc”表示 Taken；而“bc-”表示 Not Taken。

CPU 的分支预测单元在分析转移指令时可以预先得知该指令的转移结果。目前在多数 CPU 中提供了动态预测机制，而且动态预测的结果较为准确。因此在实现中，许多 PowerPC 内核并不支持静态预测机制，如 E500 内核。

CPU 使用的动态预测机制是本节研究的重点。而在不同的处理器中，分支预测单元使用的动态预测算法并不相同。在一些功能较弱的处理器，如 8 b/16 b 微控制器中，分支指令的动态预测机制较为简单。在这些处理器中，分支预测单元常使用以下几种方法动态预测分支指令的执行。

1) 分支预测单元每一次都将转移指令预测为 Taken，采用这种方法无疑非常简单，而且命中率在 50% 以上，因为无条件转移指令都是 Taken，当然使用这种方法的缺点也是显而易见的。

2) 分支预测单元将向上跳转的指令预测为 Taken，而将向下跳转的指令预测为 Not Taken。分支预测单元使用的这种预测方式与多数程序的执行风格类似，但是这种实现方式并不理想。

3) 一条转移指令被预测为 Taken，而之后这条转移指令的预测值与上一次转移指令的执行结果相同。

当采用以上几种方法时，分支预测单元的硬件实现代价较低，但是使用这些算法时，预测成功的概率较低。因此在高性能处理器中，如 PowerPC 和 x86 处理器并不会采用以上这 3 种方法实现分支预测单元。

目前在高性能处理器中，常使用 BTB (Branch Target Buffer) 管理分支预测指令。在 BTB 中含有多个 Entry，这些 Entry 由转移指令的地址低位进行索引，而这个 Entry 的 Tag 字段存放转移指令的地址高位。BTB 的功能相当于存放转移指令的 Cache，其状态机转换也与 Cache 类似。当分支预测单元第一次分析一条分支指令时，将在 BTB 中为该指令分配一个 Entry，同时也可能会淘汰 BTB 中的 Entry。目前多数处理器使用 LRU (Least Recently Used) 算法淘汰 BTB 中的 Entry。

在 BTB 的每个 Entry 中存在一个 Saturating Counter。该计数器也被称为 Bimodal Predictor，由两位组成，可以表示 4 种状态，为 0b11 时为“Strongly Taken”；为 0b10 时为“Weakly Taken”；为 0b01 时为“Weakly not Taken”；为 0b00 时为“Strongly not Taken”。

当 CPU 第一次预测一条转移指令的执行时，其结果为 Strongly Taken，此时 CPU 将在 BTB 中为该指令申请一个 Entry，并置该 Entry 的 Saturating Counter 为 0b11。此后该指令将按

⊖ 为简便起见，下文将转移指令成功进行转移称为“Taken”；而将不进行转移称为“Not Taken”。

照 Saturating Counter 的值，预测执行，如果预测结果与实际执行结果不同，将 Saturating Counter 的值减 1，直到其值为 0b00；如果相同，将 Saturating Counter 的值加 1，直到其值为 0b11。目前 Power E500 内核和 Pentium 处理器使用这种算法进行分支预测。

使用 Saturating Counter 方法在处理转移指令的执行结果为 1111011111 或者 0000001000 时的效果较好，即执行结果大多数为 0 或者 1 时的预测结果较好。然而如果一条转移指令的执行结果具有某种规律，如为 010101010101 或者 001001001001 时，使用 Saturating Counter 并不会取得理想的预测效果。

在程序的执行过程中，一条转移指令在执行过程中出现这样有规律的结果较为常见，因为程序就是按照某种规则编写的，按照某种规则完成指定的任务。为此 Two-Level 分支预测方法应运而生。

Two-Level 分支预测方法使用了两种数据结构，一种是 BHR (Branch History Register)；而另一种是 PHT (Pattern History Table)。其中 BHR 由 k 位组成，用来记录每一条转移指令的历史状态，而 PHT 表含有 2^k 个 Entry，每个 Entry 由两位 Saturating Counter 组成。BHR 和 PHT 的关系如图 3-10 所示。

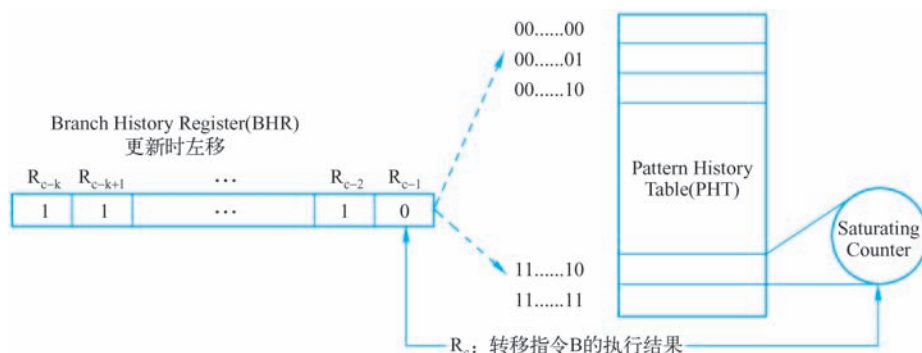


图 3-10 BHR 和 PHT 之间的关系

假设分支预测单元在使用 Two-Level 分支预测方法时，设置了一个 PBHT 表 (Per-address Branch History Table) 存放不同指令所对应的 BHR。在 PBHT 表中所有 BHR 的初始值为全 1，而在 PHT 表中所有 Entry 的初始值为 0b11。BHR 在 PBHT 表中的使用方法与替换机制与 Cache 类似。

当分支预测单元分析预测转移指令 B 的执行时，将首先从 PBHT 中获得与转移指令 B 对应的 BHR，此时 BHR 为全 1，因此 CPU 将从 PHT 的第 11...11 个 Entry 中获得预测结果 0b11，即 Strongly Taken。转移指令 B 执行完毕后，将实际执行结果 R_C 更新到 BHR 寄存器中，并同时更新 PHT 中对应的 Entry。

当 CPU 再次预测转移指令 B 的执行时，仍将根据 BHR 索引 PHT 表，并从对应 Entry 中获得预测结果。而当指令 B 再次执行完毕后，将继续更新 BHR 和 PHT 表中对应的 Entry。当转移指令的执行结果具有某种规律 (Pattern) 时，使用这种方法可以有效提高预测精度。如果转移指令 B 的实际执行结果为 001001001...001，而且 k 等于 4 时，CPU 将以 0010-0100-1001 这样的循环访问 BHR，因此 CPU 将分别从 PHT 表中的第 0010、0100 和 1001 个 Entry 中获得准确的预测结果。

由以上描述可以发现，Two-Level 分支预测法具有学习功能，并可以根据转移指令的历史记录产生的模式，在 PHT 表中查找预测结果。该算法由 T. Y. Yeh 与 Y. N. Patt 在 1991 年提出，并在高性能处理器中得到了大规模应用。

Two-Level 分支预测法具有许多变种。目前 x86 处理器主要使用“Local Branch Prediction”和“Global Branch Prediction”两种算法。

在“Local Branch Prediction”算法中，每一个 BHR 使用不同的 PHT 表，Pentium II 和 Pentium III 处理器使用这种算法。该算法的主要问题是当 PBHT 表的 Entry 数目增加时，PHT 表将以指数速度增长，而且不能利用其他转移指令的历史信息进行分支预测。而在“Global Branch Prediction”算法中，所有 BHR 共享 PHT 表，Pentium M、Pentium Core 和 Core 2 处理器使用这种算法。

在高性能处理器中，分支预测单元对一些特殊的分支指令如“Loop”和“Indirect 跳转指令”设置了“Loop Prediction”和“Indirect Prediction”部件优化这两种分支指令的预测。此外，分支预测单元还设置了 RSB（Return Stack Buffer），当 CPU 调用一个函数时，RSB 将记录该函数的返回地址，当函数返回时，将从 RSB 中获得返回地址，而不必从堆栈中获得返回地址，从而提高了函数返回的效率。

目前在高性能处理器中，动态分支预测的主要实现机制是 CPU 通过学习以往历史信息，并进行预测，因而 Neural branch predictors 机制被引入，并取得了较为理想的效果，本节对这种分支预测技术不做进一步说明。目前指令的动态分支预测技术较为成熟，在高性能计算机中，分支预测的成功概率在 95%~98% 之间，而且很难进一步提高。

3.4.2 数据预读

数据预读是指在处理器进行运算时，提前通知存储器系统将运算过程中需要的数据准备好，而当处理器需要这些数据时，可以直接从这些预读缓冲（通常指 Cache）中获得这些数据。Steven P. Vanderwiel 与 David J. Lilja 总结了最近出现的各类数据预读机制，下面将以图 3-11 为例进一步探讨这些数据预读机制。

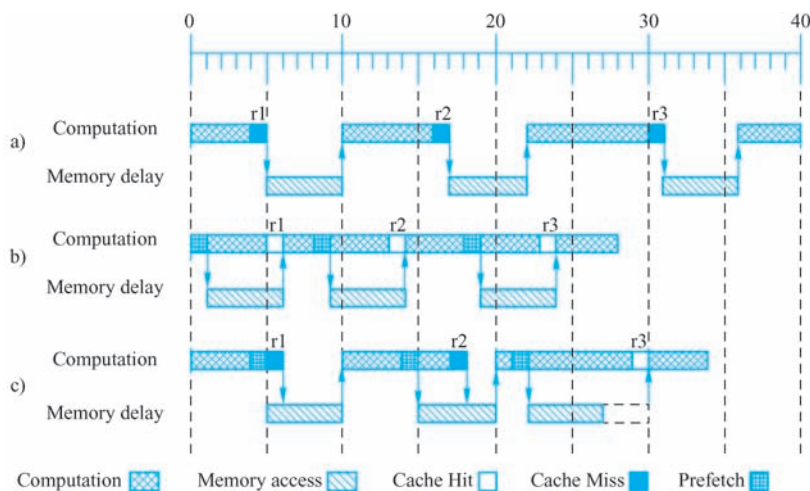


图 3-11 数据预读机制示意图

图 3-11 列举了三个实例说明数据预读的作用。其中实例 a 没有使用预读机制；实例 b 是一个采用预读机制的理想情况；而实例 c 是一个采用预读机制的次理想情况。我们假设处理器执行某个任务需要经历四个阶段，每个阶段都由处理器执行运算指令和存储指令组成。而处理器一次存储器访问需要 5 个时钟周期。其中每一个阶段的定义如下所示。

- 1) 处理器执行 4 个时钟周期后需要访问存储器。
- 2) 处理器执行 6 个时钟周期后需要访问存储器。
- 3) 处理器执行 8 个时钟周期后需要访问存储器。
- 4) 处理器执行 4 个时钟周期后完成。

实例 a 由于没有使用预读机制，因此在运算过程中需要使用存储器中的数据时，不可避免地出现 Cache Miss。实例 a 执行上述任务的过程如下。

- 1) 执行第一阶段任务的 4 个时钟周期，之后访问存储器，此时将发生 Cache Miss。
- 2) Cache Miss 需要使用一个时钟周期^①，然后在第 5 个时钟周期启动存储器读操作。
- 3) 在第 10 个周期，处理器从存储器获得数据，继续执行第二阶段任务的 6 个时钟周期，之后访问存储器，此时也将发生 Cache Miss。
- 4) 处理器在第 17~22 时钟周期从存储器读取数据，并在第 22 个时钟周期继续执行第三阶段任务的 8 个时钟周期，之后访问存储器，此时也将发生 Cache Miss。
- 5) 处理器在第 31~36 时钟周期从存储器读取数据，并在第 36 个时钟周期继续执行第四阶段任务的 4 个时钟周期，完成整个任务的执行。

采用这种机制执行上述任务共需 40 个时钟周期。而使用预读机制，可以有效缩短整个执行过程，如图 3-11 中的实例 b 所示。在实例 b 中在执行过程中，都会提前进行预读操作，虽然这些预读操作也会占用一个时钟周期，但是这些预读操作是值得的。合理使用这些数据预读，完成同样的任务 CPU 仅需要 28 个时钟周期，从而极大提高了程序的执行效率，其执行过程如下。

1) 首先使用预读指令对即将使用的存储器进行预读^②，然后执行第一阶段任务的 4 个时钟周期。当处理器进行存储器读时，数据已经准备好，处理器将在 Cache 中获得这个数据然后继续执行^③。

2) 处理器在执行第二阶段的任务时，先执行 2 个时钟周期之后进行预读操作，最后执行剩余的 4 个时钟周期。当处理器进行存储器读时，数据已经准备好，处理器将在 Cache 中获得这个数据然后继续执行。

3) 处理器执行第三阶段的任务时，先执行 4 个时钟周期之后进行预读操作，最后执行剩余的 4 个时钟周期。当处理器进行存储器读时，数据已经准备好，处理器将在 Cache 中获得这个数据然后继续执行。

4) 处理器执行第四阶段的任务，执行完 4 个时钟周期后，完成整个任务的执行。

当然这种情况是非常理想的，处理器在执行整个任务时，从始至终是连贯的，处理器执行和存储器访存完全并行，然而这种理想情况并不多见。

① 假定从访问 Cache 到发现 Cache Miss 需要一个时钟周期。

② PowerPC 处理器使用 dcbt 指令，而 x86 处理器使用 PREFETCHh 指令，实现这种软件预读。

③ 假定从 Cache 中获得数据需要一个时钟周期。

首先在一个任务的执行过程中，并不易确定最佳的预读时机；其次采用预读所获得数据并不一定能够被及时利用，因为在程序执行过程中可能会出现各种各样的分支选择，有时预读的数据并没有被及时使用。

在图 3-11 所示的实例 c 中，预读机制没有完全发挥作用，所以处理器在执行任务时，Cache Miss 时有发生，从而降低了整个任务的执行效率。即便这样，实例 c 也比完全没有使用预读的实例 a 的任务执行效率高一些。在实例 c 中，执行完毕图 3-11 中所示的任务共需要 34 个时钟周期。

但是在某些特殊情况下，采用预读机制有可能会降低效率。首先在一个较为复杂的应用中，很有可能预读的数据没有被充分利用，一个程序可能会按照不同的分支执行，而执行每一个分支所使用的数据并不相同。其次预读的数据即使是有效的，这些预读的数据也会污染整个 Cache 资源，在大规模并行任务的执行过程中，有可能引发 Cache 颠簸，从而极大地降低系统效率。

什么时候采用预读机制，关系到处理器系统结构的每一个环节，需要结合软硬件资源统筹考虑，并不能一概而论。处理器提供了必备的软件和硬件资源用以实现预读，而如何“合理”使用预读机制是系统程序员考虑的一个细节问题。数据预读可以使用软件预读或者硬件预读两种方式实现，下文将详细介绍这两种实现方式。

3.4.3 软件预读

软件预读机制由来已久，首先实现预读指令的处理器是 Motorola 的 88110 处理器，这颗处理器首先实现了“touch load”指令，这条指令是 PowerPC 处理器 dcbt 指令^①的雏形。88110 处理器是 Motorola 第一颗 RISC 处理器，具有里程碑意义。这颗处理器从内核到外部总线的设计都具有许多亮点，是 Motorola 对 PowerPC 架构做出的巨大贡献。PowerPC 架构中著名的 60X 总线也源于 88110 处理器。

后来绝大多数处理器都采用这类指令进行软件预读，Intel 在 i486 处理器中提出了 Dummy Read 指令，这条指令也是后来 x86 处理器中 PREFETCHh 指令^②的雏形。

这些软件预读指令都有一个共同的特点，就是在处理器真正需要数据之前，向存储器发出预读请求，这个预读请求不需要等待数据真正到达存储器之后，就可以执行完毕^③。从而处理器可以继续执行其他指令，以实现存储器访问与处理器运算同步进行，从而提高了程序的整体执行效率。由此可见，处理器采用软件预读可以有效提高程序的执行效率。下面考虑源代码 3-1 所示的实例。

源代码 3-1 没有采用软件预读机制的程序

```
int ip, a[N], b[N];  
  
for (i=0; i<N; i++)  
    ip=ip+a[i] * b[i];
```

① dcbt 指令是 PowerPC 处理器的一条存储器预读指令，该指令可以将内存中的数据预读到 L1 或者 L2 Cache 中。

② PREFETCHh 指令是 x86 处理器的一条存储器预读指令。

③ 预读指令在一个时钟周期内就可以执行完毕。

这个例子在对数组进行操作时被经常使用，这段源代码的作用是将 int 类型的数组 a 和数组 b 的每一项进行相乘，然后赋值给 ip，其中数组 a 和 b 都是 Cache 行对界的。源代码 3-1 中的程序并没有使用预读机制进行优化，因此这段程序在执行时会因为 a[i] 和 b[i] 中的数据不在处理器的 Cache 中，而必须启动存储器读操作。因此在这段程序在执行过程中，必须要等待存储器中的数据后才能继续，从而降低了程序的执行效率。为此将程序进行改动，如源代码 3-2 所示。

源代码 3-2 采用软件预读机制的程序

```
int ip, a[N], b[N];

for (i=0; i<N; i++) {
    fetch(&a[i+1]);
    fetch(&b[i+1]);
    ip=ip+a[i] * b[i];
}
```

以上程序对变量 ip 赋值之前，首先预读数组 a 和 b，当对变量 ip 赋值时，数组 a 和 b 中的数据已经在 Cache 中，因而不需要进行再次进行存储器操作，从而在一定程度上提高了代码的执行效率。以上代码仍然并不完美，首先，ip、a[0] 和 b[0] 并没有被预读，其次，在一个处理器中预读是以 Cache 行为单位进行的，因此对 a[0]、a[1] 进行预读时都是对同一个 Cache 行进行预读[⊖]，从而这段代码对同一个 Cache 行进行了多次预读，结果影响了执行效率。为此再次改动程序，如源代码 3-3 所示。

源代码 3-3 软件预读机制的改进程序 1

```
int ip, a[N], b[N];

fetch(&ip);
fetch(&a[0]);
fetch(&b[0]);

for (i=0; i<N-4; i+=4) {
    fetch(&a[i+4]);
    fetch(&b[i+4]);
    ip=ip+a[i] * b[i];
    ip=ip+a[i+1] * b[i+1];
    ip=ip+a[i+2] * b[i+2];
    ip=ip+a[i+3] * b[i+3];
}

for (; i<N; i++)
    ip=ip+a[i] * b[i];
```

⊖ 假定这个处理器系统的 Cache 行长度为 4 个双字，即 128 位。

对于以上例子，采用这种预读方法可以有效提高执行效率，对此有兴趣的读者可以对以上几个程序进行简单的对比测试。但是提醒读者注意，有些较为先进的编译器，可以自动加入这些预读语句，程序员不必手工加入这些预读指令。实际上源代码 3-3 中的程序还可以进一步优化。这段程序的最终优化如源代码 3-4 所示。

源代码 3-4 软件预读机制的改进程序 2

```
int ip, a[N], b[N];

fetch( &ip);

for (i=0; i<12; i+=4) {
    fetch( &a[i]);
    fetch( &b[i]);
}

for (i=0; i<N-12; i+=4) {
    fetch( &a[i+12]);
    fetch( &b[i+12]);
    ip=ip+a[i] * b[i];
    ip=ip+a[i+1] * b[i+1];
    ip=ip+a[i+2] * b[i+2];
    ip=ip+a[i+3] * b[i+3];
}

for ( ; i<N; i++)
    ip=ip+a[i] * b[i];
```

还可以对 ip、数据 a 和 b 进行充分预读之后，再一边预读数据，一边计算 ip 的值，最后计算 ip 的最终结果。使用这种方法可以使数据预读和计算充分并行，从而优化了整个任务的执行时间。

由以上程序可以发现，采用软件预读机制可以有效地对矩阵运算进行优化，因为矩阵运算进行数据访问时非常有规律，便于程序员或编译器进行优化，但是并不是所有程序都能如此方便地使用软件预读机制。此外，预读指令本身也需要占用一个机器周期，在某些情况下，采用硬件预读机制更为合理。

3.4.4 硬件预读

采用硬件预读的优点是不需要软件进行干预，也不需要浪费一条预读指令来进行预读。但硬件预读的缺点是预读结果有时并不准确，有时预读的数据并不是程序执行所需要的。在许多处理器中这种硬件预读通常与指令预读协调工作。硬件预读机制的历史比软件预读更为久远，在 IBM 370/168 处理器系统中就已经支持硬件预读机制。

大多数硬件预读仅支持存储器到 Cache 的预读，并在程序执行过程中，利用数据的局部

性原理进行硬件预读。其中最为简单的硬件预读机制是 OBL (One Block Lookahead) 机制, 采用这种机制, 当程序对数据块 b 进行读取出现 Cache Miss 时, 将数据块 b 从存储器更新到 Cache 中, 同时对数据块 $b+1$ 也进行预读并将其放入 Cache 中; 如果数据块 $b+1$ 已经在 Cache 中, 则不进行预读。

这种 OBL 机制有很多问题, 一个程序可能只使用数据块 b 中的数据, 而不使用数据块 $b+1$ 中的数据, 在这种情况下, 采用 OBL 预读机制没有任何意义。而且使用这种预读机制时, 每次预读都可能伴随着 Cache Miss, 这将极大地影响效率。有时预读的数据块 $b+1$ 会将 Cache 中可能有用的数据替换出去, 从而造成 Cache 污染。有时仅预读数据块 $b+1$ 可能并不足够, 有可能程序下一个使用的数据块来自数据块 $b+2$ 。

为了解决 OBL 机制存在的问题, 有许多新的预读方法涌现出来, 如 “tagged 预读机制”。采用这种机制, 将设置一个 “tag 位”, 处理器访问数据块 b 时, 如果数据块 b 没有在 Cache 命中, 则将数据块 b 从存储器更新到 Cache 中, 同时对数据块 $b+1$ 进行预读并将其放入 Cache 中; 如果数据块 b 已经在 Cache 中, 但是这个数据块 b 首次被处理器使用, 此时也将数据块 $b+1$ 预读到 Cache 中; 如果数据块 b 已经在 Cache 中, 但是这个数据块 b 已经被处理器使用过, 此时不将数据块 $b+1$ 预读到 Cache 中。

这种 “tagged 预读机制” 还有许多衍生机制, 比如可以将数据块 $b+1$, $b+2$ 都预读到 Cache 中, 还可以根据程序的执行信息, 将数据块 $b-1$, $b-2$ 预读到 Cache 中。

但是这些方法都无法避免因为预读而造成的 Cache 污染问题, 于是出现了 Stream Buffer 机制。采用该机制, 处理器可以将预读的数据块放入 Stream Buffer 中, 如果处理器使用的数据没有在 Cache 中, 则首先在 Stream Buffer 中查找, 采用这种方法可以消除预读对 Cache 的污染, 但是增加了系统设计的复杂性。

与软件预读机制相比, 硬件预读机制可以根据程序执行的实际情况进行预读操作, 是一种动态预读方法; 而软件预读机制需要对程序进行静态分析, 并由编译器自动或者由程序员手工加入软件预读指令来实现。

3.4.5 PCI 总线的预读机制

在一个处理器系统中, 预读的目标设备并不仅限于存储器, 程序员还可以根据实际需要对外部设备进行预读。但并不是所有的外部设备都支持预读, 只有 “well-behavior” 存储器支持预读。处理器使用的内部存储器, 如基于 SDRAM、DDR-SDRAM 或者 SRAM 的主存储器是 “well-behavior” 存储器, 有些外部设备也是 “well-behavior” 存储器。这些 well-behavior 存储器具有以下特点。

1) 对这些存储器设备进行读操作时不会改变存储器的内容。显然主存储器具有这种性质。如果一个主存储器的一个数据为 0, 那么读取这个数据 100 次也不会将这个结果变为 1。但是在外部设备中, 一些使用存储器映像寻址的寄存器具有读清除的功能。比如某些中断状态寄存器[⊖]。当设备含有未处理的中断请求时, 该寄存器的中断状态位为 1, 对此寄存器进行读操作时, 硬件将自动地把该中断位清零, 这类采用存储映像寻址的寄存器就不是 well-behavior 存储器。

⊖ 假设中断状态寄存器支持读清除功能。

2) 对“well-behavior”存储器的多次读操作，可以合并为一次读操作。如向这个设备的地址 n , $n+4$, $n+8$ 和 $n+12$ 地址处进行四个双字的读操作，可以合并为对 n 地址的一次突发读操作（大小为 4 个双字）。

3) 对“well-behavior”存储器的多次写操作，可以合并为一次写操作。如向这个设备的地址 n , $n+4$, $n+8$ 和 $n+12$ 地址处进行四个双字的写操作，可以合并为对 n 地址的一次突发写操作。对于主存储器，进行这种操作不会产生副作用，但是对于有些外部设备，不能进行这种操作。

4) 对“well-behavior”的存储器写操作，可以合并为一次写操作。向这个设备的地址 n , $n+1$, $n+2$ 和 $n+3$ 地址处进行四个单字的写操作，可以合并为对 n 地址的一次 DW 写操作。对主存储器进行这种操作不会出现错误，但是对于有些外部设备，不能进行这种操作。

如果外部设备满足以上四个条件，该外部设备被称为“well-behavior”。PCI 配置空间的 BAR 寄存器中有一个“Prefetchable”位，该位为 1 时表示这个 BAR 寄存器所对应的存储器空间支持预读。PCI 总线的预读机制需要 HOST 主桥、PCI 桥和 PCI 设备的共同参与。在 PCI 总线中，预读机制需要分两种情况进行讨论，一个是 HOST 处理器通过 HOST 主桥和 PCI 桥访问最终的 PCI 设备；另一个是 PCI 设备使用 DMA 机制访问存储器。

PCI 总线预读机制的拓扑结构如图 3-12 所示。

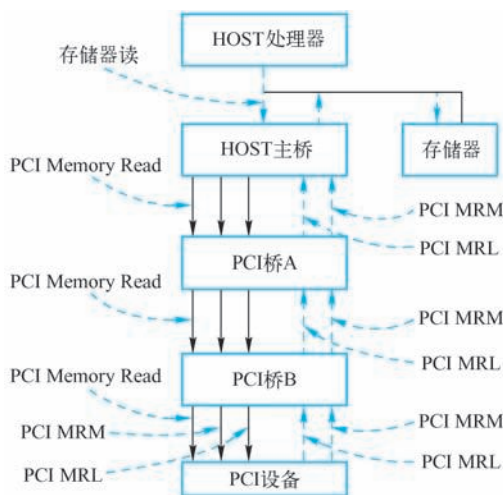


图 3-12 PCI 总线的预读

由图 3-12 可知，HOST 处理器预读 PCI 设备时，需要经过 HOST 主桥，并可能通过多级 PCI 桥，最终到达 PCI 设备，在这个数据传送路径上，有的 PCI 桥支持预读，有的不支持预读。而 PCI 设备对主存储器进行预读时也将经过多级 PCI 桥。PCI 设备除了可以对主存储器进行预读之外，还可以预读其他 PCI 设备，但是这种情况在实际应用中极少出现，本节仅介绍 PCI 设备预读主存储器这种情况。

1. HOST 处理器预读 PCI 设备

PCI 设备的 BAR 寄存器可以设置预读位，首先支持预读的 BAR 寄存器空间必须是一个 well-behavior 的存储器空间，其次 PCI 设备必须能够接收来自 PCI 桥和 HOST 主桥的 MRM

(Memory Read Multiple) 和 MRL (Memory Read Line) 总线事务。

如果 PCI 设备支持预读, 那么当处理器对这个 PCI 设备进行读操作时, 可以通过 PCI 桥启动预读机制 (该 PCI 桥也需要支持预读), 使用 MRM 和 MRL 总线事务, 对 PCI 设备进行预读, 并将预读的数据暂时存放在 PCI 桥的预读缓冲中。

之后当 PCI 主设备继续读取 PCI 设备的 BAR 空间时, 如果访问的数据在 PCI 桥的预读缓冲中, PCI 桥可以不对 PCI 设备发起存储器读总线事务, 而是直接从预读缓冲中获取数据, 并将其传递给 PCI 主设备。当 PCI 主设备完成读总线事务后, PCI 桥必须丢弃预读的数据以保证数据的完整性。此外当 PCI 桥预读的地址空间超越了 PCI 设备可预读 BAR 空间边界时, PCI 设备需要 “disconnect” 该总线事务。

如果 PCI 桥支持 “可预读” 的存储器空间, 而且其下挂接的 PCI 设备 BAR 空间也支持预读时, 系统软件需要从 PCI 桥 “可预读” 的存储器空间中为该 PCI 设备分配空间。此时 PCI 桥可以将 PCI 设备预读的数据暂存在 PCI 桥的预读缓冲中。

PCI 总线规定, 如果下游 PCI 桥地址空间支持预读, 则其上游 PCI 桥地址空间既可以支持也可以不支持预读机制。如图 3-12 所示, 如果 PCI 桥 B 管理的 PCI 子树使用了可预读空间时, PCI 桥 A 可以不支持可预读空间, 此时 PCI 桥 A 只能使用存储器读总线事务读取 PCI 设备, 而 PCI 桥 B 可以将这个存储器读总线事务转换为 MRL 或者 MRM 总线事务, 预读 PCI 设备的 BAR 空间 (如果 PCI 设备的 BAR 空间支持预读), 并将预读的数据保存在 PCI 桥 B 的数据缓冲中。

但是 PCI 总线不允许 PCI 桥 A 从其 “可预读” 的地址空间中, 为 PCI 桥 B 的 “不可预读” 区域预留空间, 因为这种情况将影响数据的完整性。

大多数 HOST 主桥并不支持对 PCI 设备的预读, 这些 HOST 主桥并不能向 PCI 设备发出 MRL 或者 MRM 总线事务。由于在许多处理器系统中, PCI 设备是直接挂接到 HOST 主桥上的, 如果连 HOST 主桥也不支持这种预读, 即便 PCI 设备支持了预读机制也没有实际作用。而且如果 PCI 设备支持预读机制, 硬件上需要增加额外的开销, 这也是多数 PCI 设备不支持预读机制的原因。

尽管如此, 本节仍需要对 HOST 处理器预读 PCI 设备进行探讨。假设在图 3-12 所示的处理器系统中, HOST 主桥和 PCI 桥 A 不支持预读, 而 PCI 桥 B 支持预读, 而且处理器的 Cache 行长度为 32B (0x20)。

如果 HOST 处理器对 PCI 设备的 0x8000-0000~0x8000-0003 这段地址空间进行读操作, HOST 主桥将使用存储器读总线事务读取 PCI 设备的 “0x8000-0000~0x8000-0003 这段地址空间”, 这个存储器读请求首先到达 PCI 桥 A, 再由 PCI 桥 A 转发给 PCI 桥 B。

PCI 桥 B 发现 “0x8000-0000~0x8000-0003 这段地址空间” 属于自己的可预读存储器区域, 即该地址区域在该桥的 Prefetchable Memory Base 定义的范围内, 则将该存储器读请求转换为 MRL 总线事务, 并使用该总线事务从 PCI 设备[⊖]中读取 0x8000-0000~0x8000-001F 这段数据, 并将该数据存放到 PCI 桥 B 的预读缓冲中。MRL 总线事务将从需要访问的 PCI 设备的起始地址开始, 一直读到当前 Cache 行边界。

[⊖] 此时 PCI 设备的这段区域一定是可预读的存储器区域。

之后当 HOST 处理器读取 0x8000-0004~0x8000-001F 这段 PCI 总线地址空间的数据时，将从 PCI 桥 B 的预读缓冲中直接获取数据，而不必对 PCI 设备进行读取。

2. PCI 设备读取存储器

PCI 设备预读存储器地址空间时，需要使用 MRL 或者 MRM 总线事务。与 MRL 总线周期不同，MRM 总线事务将从需要访问的存储器起始地址开始，一直读到下一个 Cache 行边界为止。

对于一个 Cache 行长度为 32B (0x20) 的处理器系统，如果一个 PCI 设备在对主存储器的 0x1000-0000~0x1000-0007 这段存储器地址空间进行读操作时，由于这段空间没有跨越 Cache 行边界，此时 PCI 设备将使用 MRL 总线事务对 0x1000-0000~0x1000-001F 这段地址区域发起存储器读请求。

如果一个 PCI 设备在对主存储器的 0x1000-001C~0x1000-0024 这段存储器地址空间进行读操作时，由于这段空间跨越了 Cache 行边界，此时 PCI 设备将使用 MRM 总线事务对 0x1000-001C~0x1000-002F 这段地址空间发起存储器读请求。

在图 3-12 所示的例子中，PCI 设备读取 0x1000-001C~0x1000-0024 这段存储器地址空间时，首先将使用 MRM 总线事务发起读请求，该请求将通过 PCI 桥 B 和 A 最终到达 HOST 主桥。HOST 主桥将从主存储器中读取 0x1000-001C~0x1000-002F 这段地址空间的数据^①。如果 PCI 桥 A 也支持下游总线到上游总线的预读，这段数据将传递给 PCI 桥 A；如果 PCI 桥 A 和 B 都支持这种预读，这段数据将到达 PCI 桥 B 的预读缓冲。

如果 PCI 桥 A 和 B 都不支持预读，0x1000-0024~0x1000-002F 这段数据将缓存在 HOST 主桥中，HOST 主桥仅将 0x1000-001C~0x1000-0024 这段数据通过 PCI 桥 A 和 B 传递给 PCI 设备。之后当 PCI 设备需要读取 0x1000-0024~0x1000-002F 这段数据时，该设备将根据不同情况，从 HOST 主桥、PCI 桥 A 或者 B 中获取数据而不必读取主存储器。值得注意的是，PCI 设备在完成一次数据传送后，暂存在 HOST 主桥中的预读数据将被清除。PCI 设备采用这种预读方式，可以极大提高访问主存储器的效率。

PCI 总线规范有一个缺陷，就是目标设备并不知道源设备究竟需要读取或者写入多少个数据。例如 PCI 设备使用 DMA 读方式从存储器中读取 4 KB 大小的数据时，只能通过 PCI 突发读方式，一次读取一个或者多个 Cache 行。

假定 PCI 总线一次突发读写只能读取 32B 大小的数据，此时 PCI 设备读取 4KB 大小的数据，需要使用 128 次突发周期才能完成全部数据传送。而 HOST 主桥只能一个一个地处理这些突发传送，从而存储器控制器并不能准确预知何时 PCI 设备将停止读取数据。在这种情况下，合理地使用预读机制可以有效地提高 PCI 总线的数据传送效率。

我们首先假定 PCI 设备一次只能读取一个 Cache 行大小的数据，然后释放总线，之后再读取一个 Cache 行大小的数据。如果使用预读机制，虽然 PCI 设备在一个总线周期内只能获得一个 Cache 行大小的数据，但是 HOST 主桥仍然可以从存储器获得 2 个 Cache 行以上的数据，并将这个数据暂存在 HOST 主桥的缓冲中，之后 PCI 设备再发起突发周期时，HOST 主桥可以不从存储器，而是从缓冲中直接将数据传递给 PCI 设备，从而降低了 PCI 设备对存储

① 假设 HOST 主桥读取存储器时支持预读，多数 HOST 主桥都支持这种预读。

器访问的次数，提高了整个处理器系统的效率。

以上描述仅是实现 PCI 总线预读的一个例子，而且仅仅是理论上的探讨。实际上绝大多数半导体厂商都没有公开 HOST 主桥预读存储器系统的细节，在多数处理器中，HOST 主桥以 Cache 行为单位读取主存储器的内容，而且为了支持 PCI 设备的预读功能 HOST 主桥需要设置必要的缓冲部件，这些缓冲的管理策略较为复杂。

目前 PCI 总线已经逐渐退出历史舞台，进一步深入研究 PCI 桥和 HOST 主桥，意义并不大，不过读者依然可以通过学习 PCI 体系结构，获得处理器系统中有关外部设备的必要知识，并以此为基础，学习 PCIe 体系结构。

3.5 小结

本章重点介绍了 PCI 总线的数据交换。其中最重要的内容是与 Cache 相关的 PCI 总线事务和预读机制。虽然与 Cache 相关的 PCI 总线事务并不多见，但是理解这些内容对于理解 PCI 和处理器体系结构，非常重要。

第 I 篇的主体是以 PCI 总线为例，说明一个局部总线在处理器系统中的作用，这也是笔者写作本书的初衷。PCI 总线作为一个局部总线，在设计思路，与其他局部总线并没有本质的不同。在本篇中，最重要的内容是局部总线的设计与实现方法，希望读者阅读本书时，不要仅仅将目光锁定在 PCI 总线本身。

本书的第 II 篇内容与第 I 篇密切相关，希望读者在真正理解第 I 篇内容的基础上阅读第 II 篇。PCIe 总线在继承 PCI 总线部分内容的基础上做出了许多重大调整。但是从处理器体系结构的角度来看，PCIe 总线依然是局部总线，这条局部总线与 PCI 总线以及其他平台的局部总线相比，并不存在本质的不同。而理解这些局部总线的关键，仍然是深入理解处理器的体系结构。